Pharmacy Inventory & Prescription Management System

Database Systems Lab (BCSE302P)

PROJECT REPORT

Fall Semester 2026-2027

Submitted by

ABHIRUP BANIK (24BCE0311)

SARVASWA BHANTI (24BCE0333)

AMOGH SHARMA (24BCE2126)

SAMI FARHAN SYED (24BCE2744)

in partial fulfillment for the award of the degree of

B. Tech

In

Computer Science and Engineering
Core



Vellore-632014, Tamil Nadu, India

School of Computer Science and Engineering

October, 2026

CONTENTS

Pharmacy Inventory & Prescription Management System

1. Short summary of the project

1.1 Overview of the Two-Database Design

1.2 Polyglot Persistence Approach

1.3 The Workflow, End to End

1.4 Proposed Model and Tech Stack

1.4.1 The Layered Model

1.4.2 Tech Stack

1.4.3 Complete Data Model: What Each Table Stores

1.4.4 Data Flow: What Goes Where, and Why

2. Front end and Database details

2.1 Front End Details

2.2 Database Details - Overview

2.3 Database One - PostgreSQL: The Transactional Core

2.3.1 The FEFO Query

2.3.2 The Dispensing Event / Audit Log Table

2.3.3 The Dispense Transaction as Implemented

2.4 Database Two - MongoDB: The Safety Knowledge Layer

2.4.1 Sample Interaction Document

2.4.2 Interaction Data as Built

2.5 Comparison of the Two Databases

3. Implementation and Results

3.1 Changes Since the Design Review

3.2 Database Objects as Built

3.3 Measured Results

3.4 Running the Project

4. Demonstration

4.1 Creating a Prescription

4.2 Dispensing with an Interaction Warning

4.3 Unsafe Dispenses Are Refused

4.4 A Request Larger Than One Batch

4.5 Receiving Stock and the Inventory View

4.6 The Audit Log and the Weekly Report

4.7 Drug Interactions in MongoDB

4.8 Two Pharmacists, the Last Unit

4.9 MongoDB Unreachable Mid-Dispense

4.10 Why the FEFO Query Stays Fast

4.11 The Audit Log Cannot Be Rewritten

PHARMACY INVENTORY & PRESCRIPTION MANAGEMENT SYSTEM

1. SHORT SUMMARY OF THE PROJECT

The project is a pharmacy management system in which the database layer, rather than the user interface, is the centre of the design. A pharmacy has to do two things correctly at the moment a medicine is handed over: it has to deduct the right physical stock from the right batch without ever losing count, and it has to confirm that the medicine being dispensed does not conflict dangerously with the other medicines on the same prescription. These two requirements pull in different directions, and the system is built around that observation.

1.1 Overview of the Two-Database Design

When a pharmacist dispenses a medicine, PostgreSQL handles the transaction, picks the right batch and logs the event, while MongoDB checks whether the combination is safe. Each database is responsible for the part of the task the other is not well suited to.

1.2 Polyglot Persistence Approach

Polyglot persistence means picking a different database for each part of a system based on how that data actually behaves, instead of forcing everything into one engine. Here, inventory data needs strong consistency and locking because it involves real stock and real money, and it also needs a durable, ordered record of what was dispensed and when. Interaction data, by contrast, changes shape often, arrives from external medical sources and is read far more often than it is written, so a flexible document store fits it better than a rigid table definition.

The dispensing event log is deliberately kept inside PostgreSQL as an ordinary relational audit table rather than being pushed into a third engine. Writing it alongside the stock update means the audit trail is committed under the same ACID guarantees as the transaction it describes, and a single well-indexed table answers the queries the system actually needs without the added operational overhead of running and maintaining a third database.

Figure 1 shows the overall architecture, with the two engines and the responsibilities assigned to each.

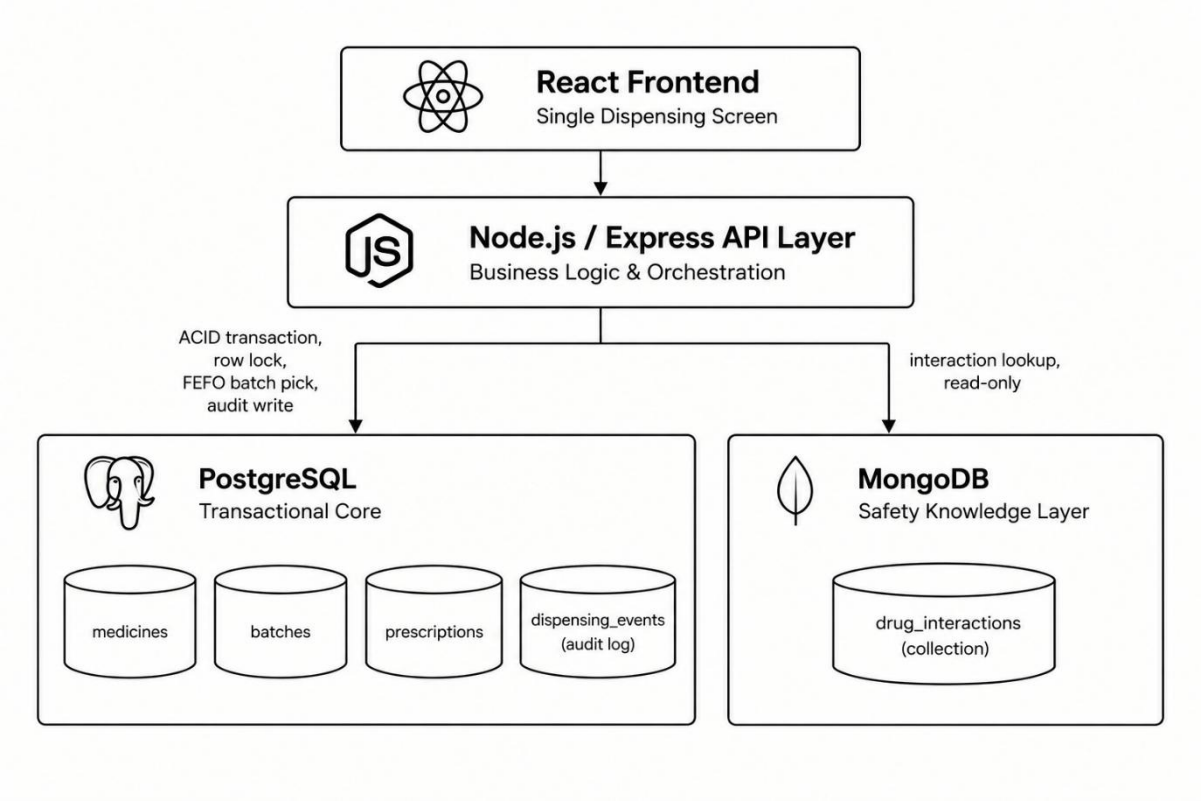


Figure 1: System architecture of the two-database polyglot persistence design

1.3 The Workflow, End to End

One dispensing action touches both databases, each for a different reason. The sequence is set out below and illustrated in Figure 2.

1. The pharmacist searches for a medicine against a prescription.
2. PostgreSQL locks the relevant batch row and selects the batch nearest to expiry, following the FEFO (first expired, first out) rule.
3. The backend queries MongoDB for interactions between all medicines on that prescription.
4. If a conflict is found, a warning is shown before the pharmacist is allowed to proceed.
5. On confirmation, PostgreSQL commits the transaction and updates the stock quantity for that batch.
6. PostgreSQL writes an entry to the dispensing_events (audit_log) table as part of the same transaction, so the event record and the stock change succeed or fail together.

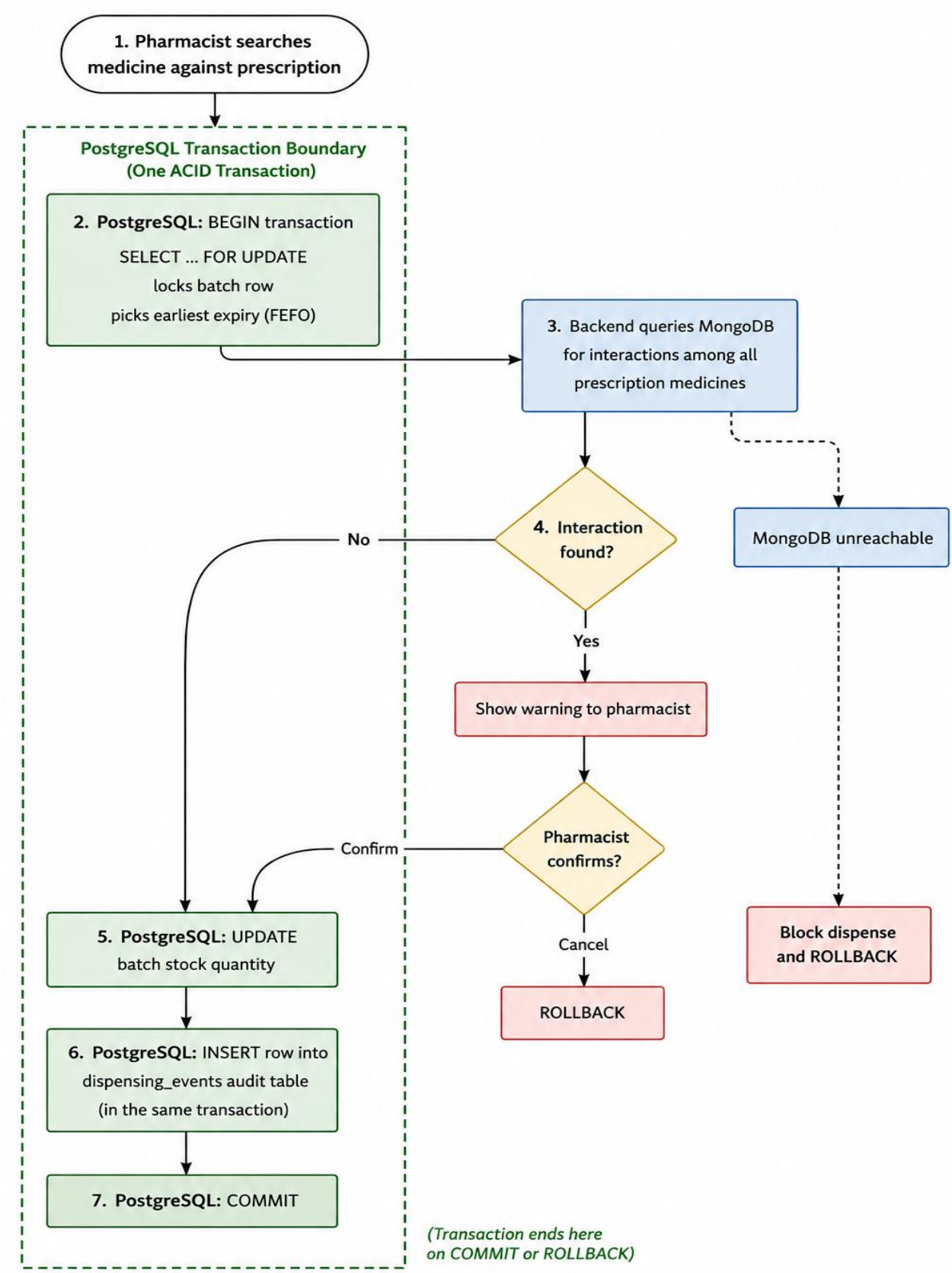


Figure 2: End-to-end dispensing workflow across PostgreSQL and MongoDB

As built, the order of steps 2 to 6 changed in one important way: the batch row is not locked while the pharmacist reads the warning. Searching and checking happen first with no lock at all, and only pressing Confirm opens the short transaction that locks, re-checks, updates and logs. Section 2.3.3 and Figure 9 show the implemented sequence; Section 3.1 lists every change made since the design review.

1.4 Proposed Model and Tech Stack

This section expands the architecture of Figure 1 into the full model that was built: the three layers and the technology in each, every table and the data it holds, and, for each action a pharmacist takes, which table or collection the data goes to and why. Figure 3, Figure 4 and Figure 5 are read in that order.

1.4.1 The Layered Model

The application has three layers. The pharmacist only ever talks to the browser; the browser only ever talks to the API; and only the API talks to the two databases. Every rule that matters is therefore enforced on the server or inside the database, never in the browser.

- **Presentation layer.** React 18 screens for dispensing, new prescriptions, inventory, the audit log, interactions, and an "Under the hood" screen that runs the database demonstrations live.
- **API layer.** Node.js with Express. dispense.js holds the two-phase dispense (preview, then confirm), and db.js holds two PostgreSQL connection pools, one per role, and the MongoDB client with a 1.5 second timeout.
- **Data layer.** PostgreSQL owns everything that must be exact: stock, prescriptions and the audit history. MongoDB owns the drug-interaction knowledge, whose shape varies from pair to pair.

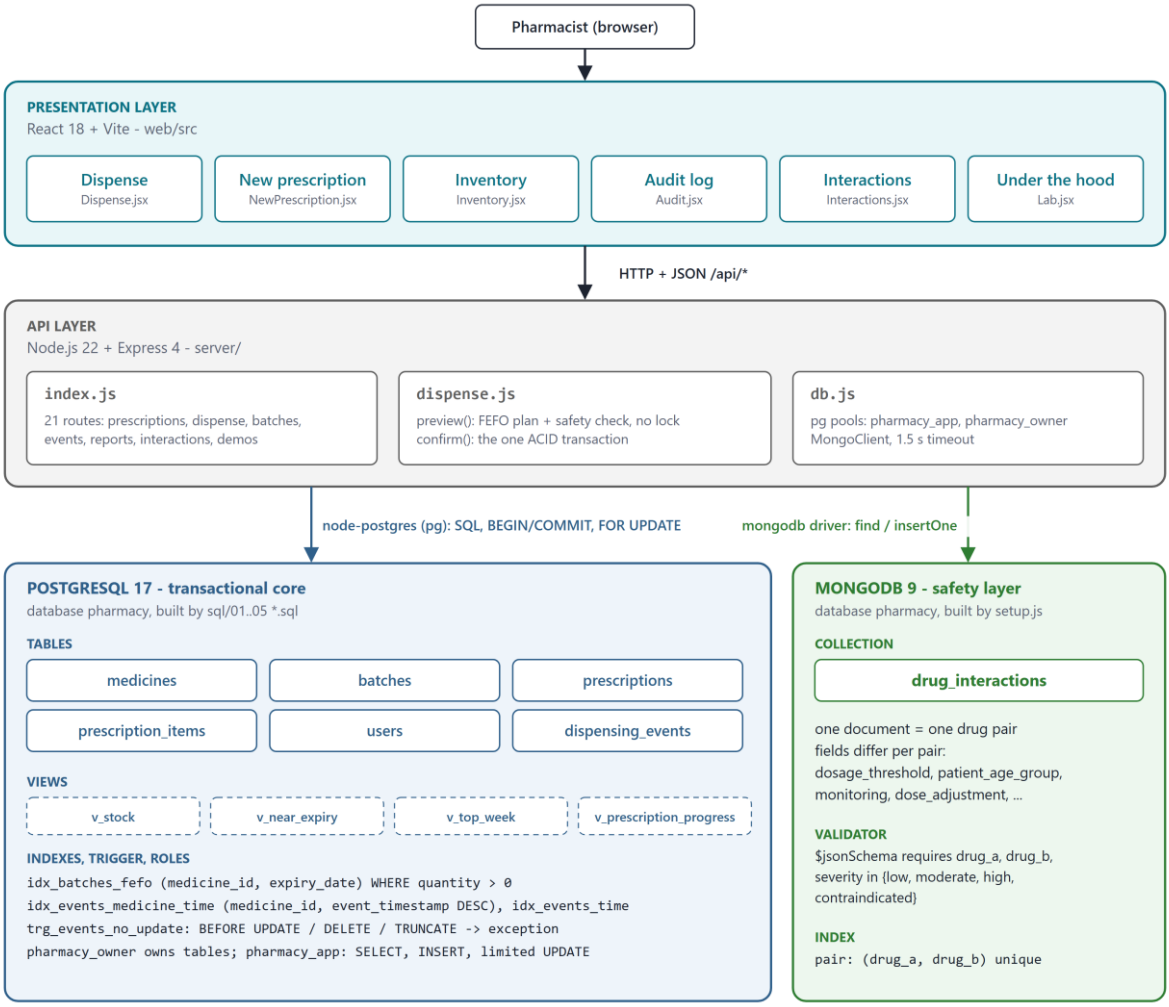


Figure 3: Proposed model: three layers, the tech stack and the database objects in each engine

1.4.2 Tech Stack

Each technology was chosen for one job. The versions are the ones the project runs on.

Layer	Technology	Why it is used
Transactional database	PostgreSQL 17.11	ACID transactions, SELECT ... FOR UPDATE row locks, CHECK and foreign-key constraints, partial B-tree indexes, triggers and role privileges
Safety database	MongoDB 9.0.2	Interaction documents with a different set of fields per drug pair; $jsonSchema validator for the required fields; unique compound index on the pair

Layer	Technology	Why it is used
API server	Node.js 22 + Express 4	One process serves the 21 API routes and the built web app on one address
Database drivers	node-postgres (pg) 8, mongodb driver 6	pg gives explicit BEGIN / COMMIT / ROLLBACK on one client; the MongoDB driver gives a bounded server-selection timeout
Front end	React 18 + Vite 6	Component screens for each task; Vite builds a static bundle that Express serves
Tooling	setup.js, smoke.js	setup.js rebuilds both databases from scratch in about two seconds; smoke.js runs 24 end-to-end checks against the running API

1.4.3 Complete Data Model: What Each Table Stores

Figure 4 expands the ER diagram of Figure 7 to every column, type and constraint as built, together with the trigger, the views, the indexes and the MongoDB collection. The two databases meet at a single value: medicines.generic_name in PostgreSQL is the drug_a / drug_b key in MongoDB.

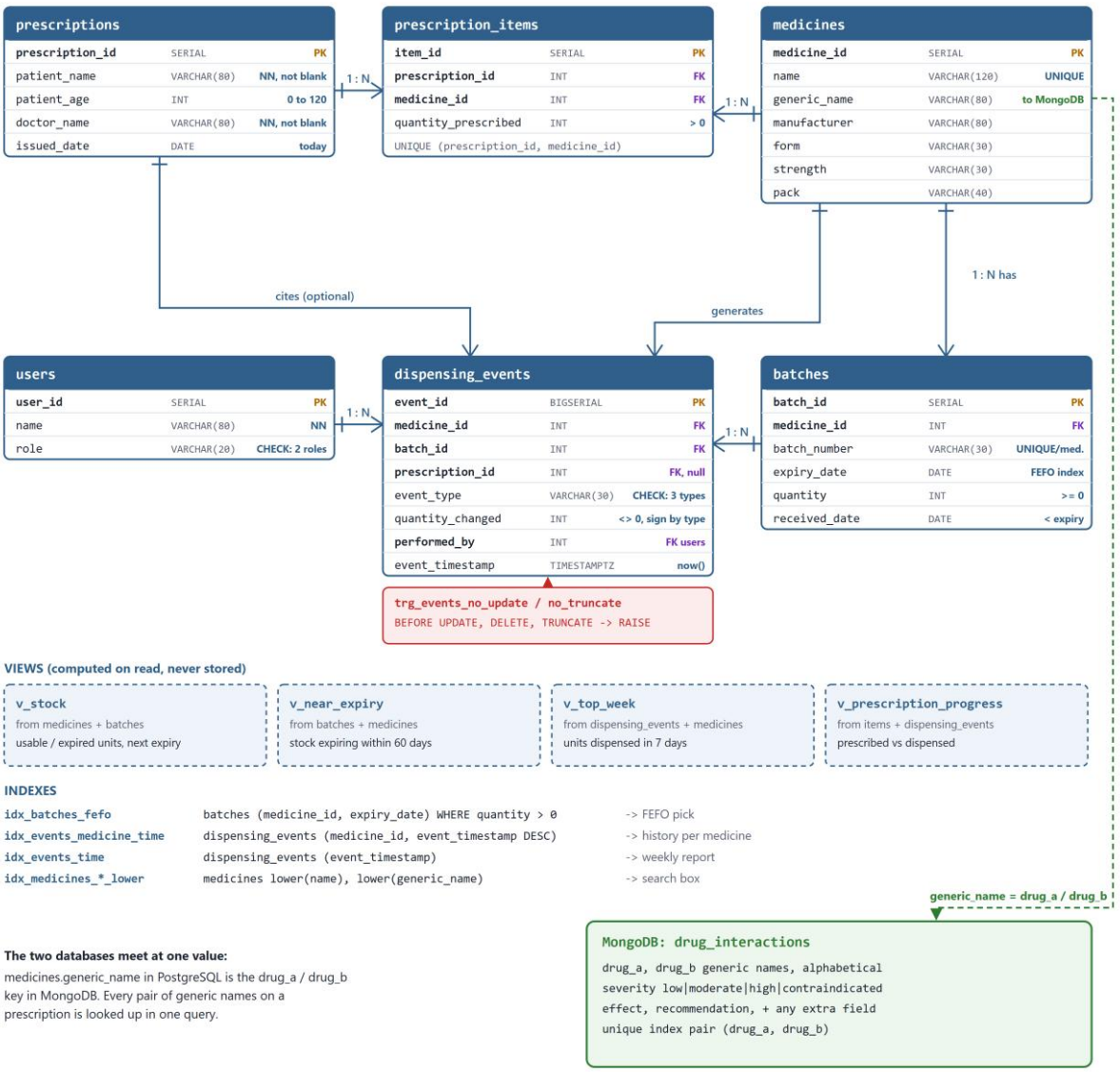


Figure 4: Complete data model: PostgreSQL tables, trigger, views and indexes, and the MongoDB collection

What each table holds, and which actions write and read it:

Table	What it stores	Written by / read by
medicines	One row per product: name, generic name, form, strength, pack, manufacturer. generic_name is the key MongoDB is searched by.	Seeded once. Read by search, every preview and every report.

Table	What it stores	Written by / read by
batches	One row per physical lot: batch number, expiry date, quantity on hand, date received. This is the source of truth for stock.	Inserted on receipt; quantity reduced on dispense under a row lock; read by the FEFO query.
prescriptions	The header of a prescription: patient name, age, doctor, date issued.	Inserted from the New prescription form; read when a prescription is opened.
prescription_items	One row per medicine on a prescription with the quantity prescribed.	Inserted with the prescription in one transaction; locked on confirm so a line cannot be over-dispensed.
users	The pharmacists and the admin who perform actions.	Seeded. Referenced by every audit row as performed_by.
dispensing_events	Append-only history: every DISPENSE, RECEIPT and ADJUSTMENT with batch, signed quantity change, prescription, who and when.	Inserted in the same transaction as each stock change; never updated or deleted; read by the audit screen and the weekly report.
drug_interactions (MongoDB)	One document per drug pair: severity, effect, recommendation, plus any extra fields that pair needs.	Read on every preview and again inside every confirm; written rarely, from the Interactions screen.

1.4.4 Data Flow: What Goes Where, and Why

Figure 5 answers the question of what data goes where. Each row is one thing the pharmacist does. Reading left to right gives the action on screen, the API call it makes, every PostgreSQL table or view it touches with the kind of access, and what it asks of MongoDB.

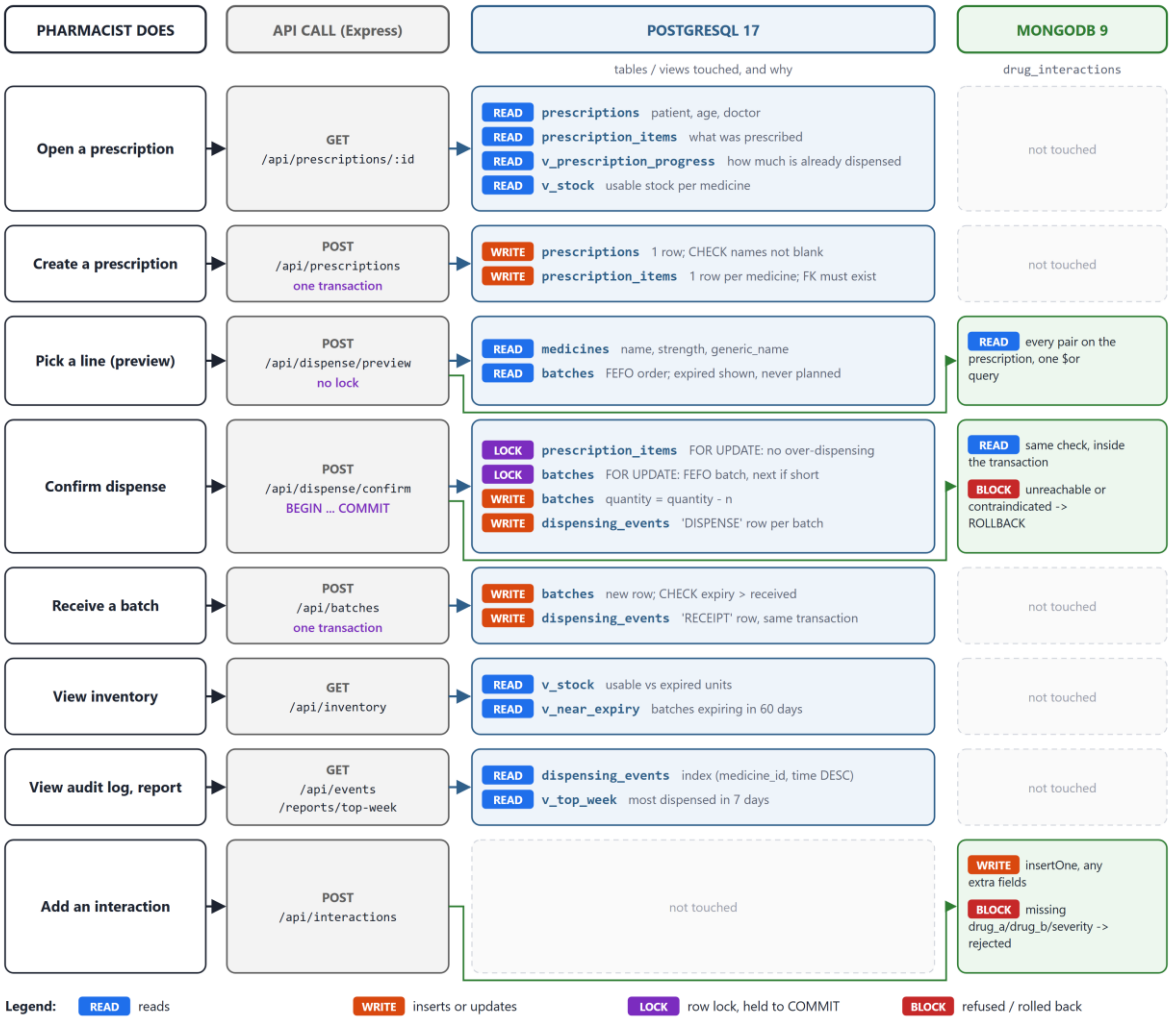


Figure 5: Data flow: for each action, the API call, the tables touched and the MongoDB use

Reading the diagram, five points explain the design:

- **Reads lock nothing.** Opening a prescription and previewing a dispense only read, so the pharmacist can look at a warning for as long as needed without blocking anyone.
- **Only Confirm takes locks, always in the same order.** prescription_items first, then batches in expiry order. Because every transaction locks in the same order, two confirms cannot deadlock each other.
- **Every stock change leaves a row.** Confirm and Receive both write batches and dispensing_events inside one transaction, so the history and the stock always agree; the sum of the audit log equals the stock on hand.
- **MongoDB is asked twice for a dispense.** Once in preview, to show the warning, and again inside the confirm transaction, so an outage or a newly added rule is caught before COMMIT and the dispense rolls back.

- **Each engine keeps to its own data.** MongoDB never holds stock and PostgreSQL never holds interaction rules; the only value they share is the generic name. The views are computed on read and store nothing.

2. FRONT END AND DATABASE DETAILS

2.1 Front End Details

The relevant Front End details are:

- A single dispensing screen: search a medicine, see batch and expiry, see any interaction warning, and confirm the dispense.
- Built in React 18 with Vite and served by the same Express server, so the whole application opens at one address, http://localhost:4000.
- A shared header shows the date, the pharmacist performing the actions and a live status light for each database, refreshed every five seconds.
- The browser calls only the API; it never talks to a database directly, so every rule is enforced on the server and inside the database.
- **Dispense.** The screen from the wireframe: choose a prescription, pick a line, see the batches with the FEFO pick starred and expired lots struck through, read the interaction warning, then Confirm or Cancel.
- **New prescription.** Patient, age, doctor and any number of medicine lines, saved in one transaction.
- **Inventory.** Stock per medicine from the v_stock view, every batch of a medicine, a form to receive a new batch, and stock expiring within 60 days.
- **Audit log.** The dispensing_events table filtered by medicine and event type, and the most dispensed medicines this week.
- **Interactions.** Every MongoDB document, and a box to insert a new one with any extra fields.
- **Under the hood.** Four live demonstrations: the concurrency race, a MongoDB outage, EXPLAIN ANALYZE of the FEFO query, and an attempt to rewrite the audit log.

A rough layout of that single screen is shown in Figure 6.

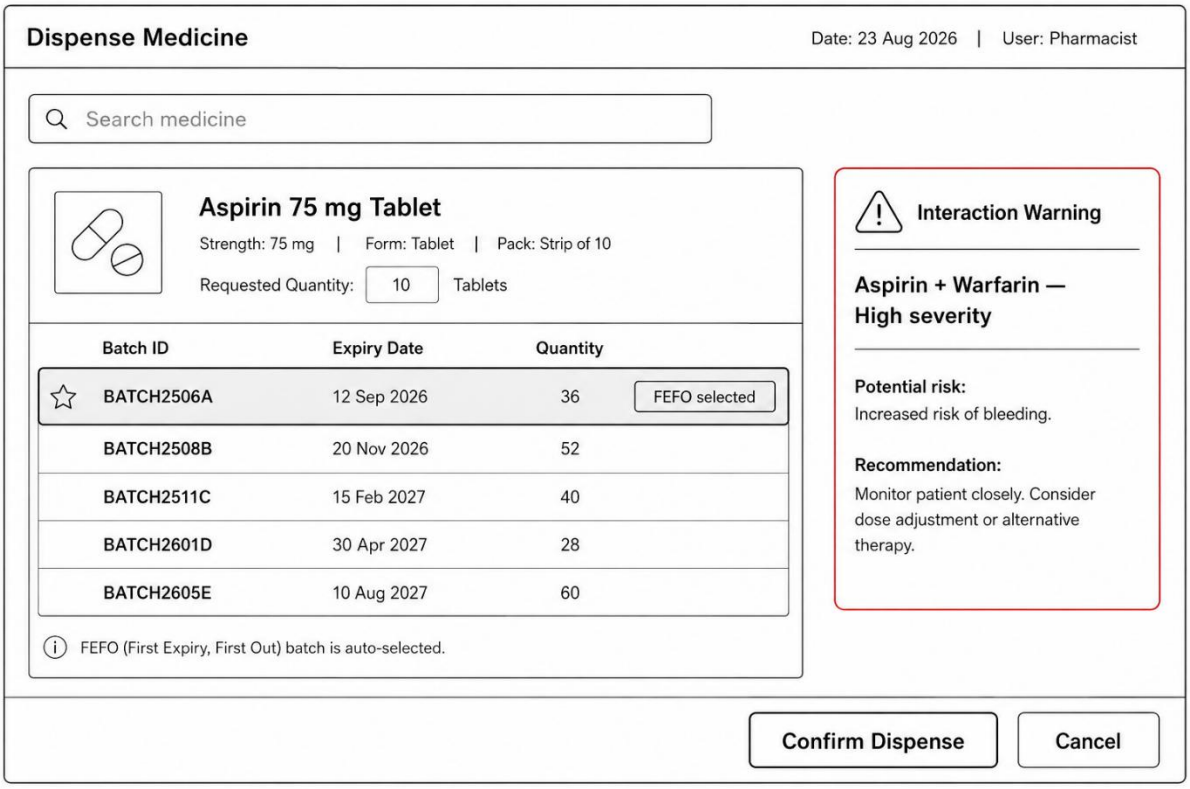


Figure 6: Wireframe of the single dispensing screen

The built Dispense screen follows this wireframe closely. Every screen is shown with real data, step by step, in Section 4.

2.2 Database Details - Overview

The system uses exactly two databases. PostgreSQL is the transactional core, holding medicines, batches, prescriptions, stock movements and the dispensing audit log. MongoDB is the safety knowledge layer, holding drug interaction records and clinical guidelines. Each is described below with the concepts it is being used to demonstrate.

2.3 Database One - PostgreSQL: The Transactional Core

Why PostgreSQL here: inventory counts and prescriptions are relational by nature and directly tied to stock and billing. If two people attempt to dispense the last unit of a batch at the same time, this concurrency issue must be prevented. PostgreSQL's ACID guarantees make this possible.

- **ACID transactions.** The whole dispense operation, from checking stock to updating quantity to writing the audit row, runs inside one transaction. If any step fails, everything rolls back and nothing is left half-updated.

- **Row-level locking.** SELECT ... FOR UPDATE locks the batch row being read. A second pharmacist trying to dispense from the same batch has to wait until the first transaction commits or rolls back.

- **Indexing for FEFO.** A B-tree index on expiry_date within each medicine's batches means the "pick the earliest expiring batch" query stays fast even as batch history grows.

- **Referential integrity.** Foreign keys tie batches to medicines and prescriptions to medicines. A batch cannot be inserted for a medicine that does not exist, and the database enforces that rather than the application code.

- **Audit logging.** A dispensing_events table records every dispense, stock adjustment and batch receipt as an append-only row, written as part of the same transaction that changes the stock.

The relational schema, including the foreign-key relationships and the audit table, is shown in Figure 7.

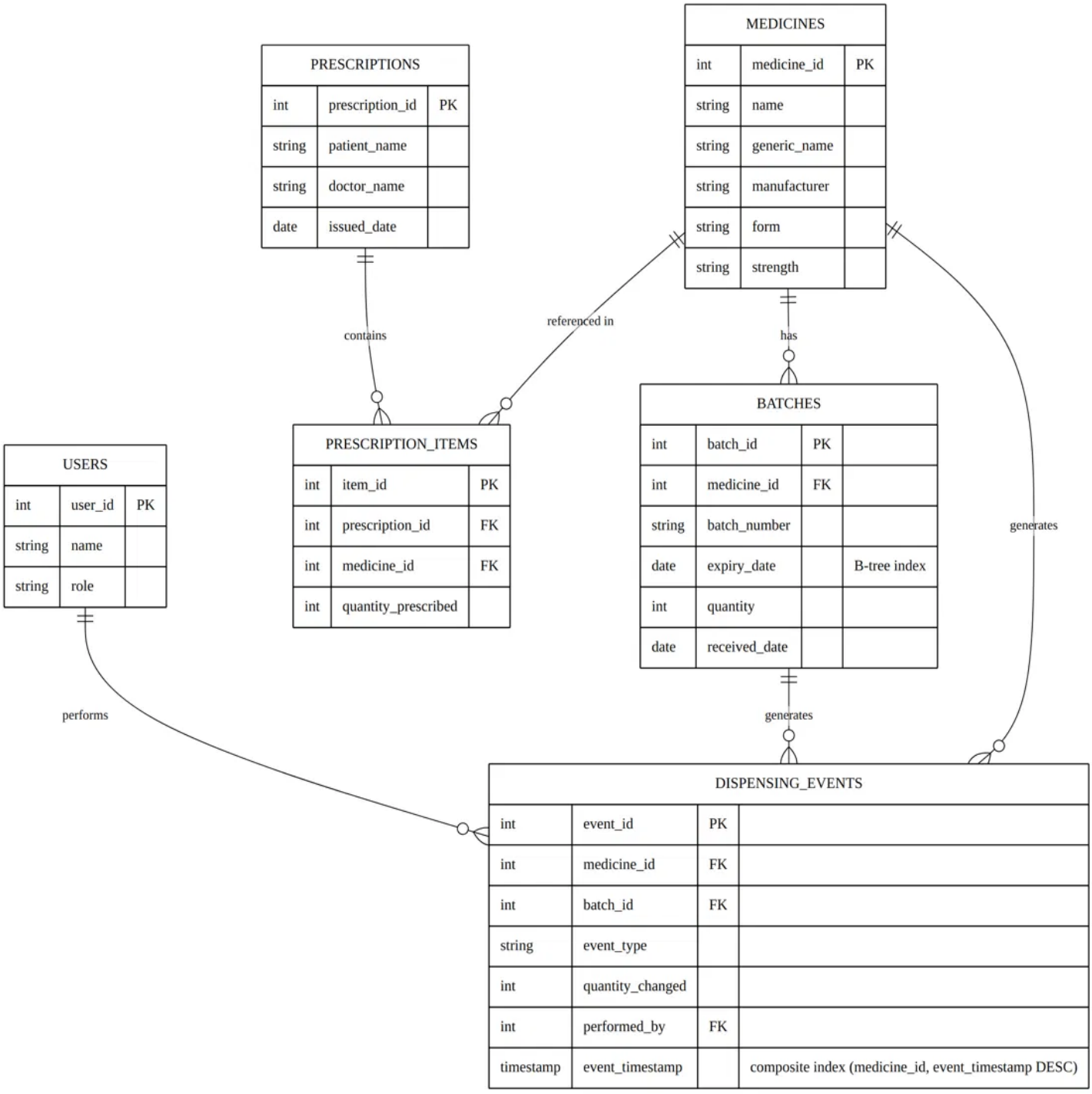


Figure 7: ER diagram of the PostgreSQL schema

2.3.1 The FEFO Query

This is the query at the centre of the transactional design. It is also the query that needs the lock, because two transactions could otherwise both read the same row before either commits.

```
SELECT batch_id, batch_number, expiry_date, quantity
FROM batches
WHERE medicine_id = $1 AND quantity > 0
AND expiry_date > current_date
AND batch_id <> ALL($2::int[])
ORDER BY expiry_date, batch_id
LIMIT 1
FOR UPDATE;
```

Two conditions were added during the build. expiry_date > current_date stops an expired batch from ever being picked; in the wireframe the starred batch had already expired. batch_id <> ALL($2) lists the batches already taken in this transaction, so when the first batch is too small the same query is run again and locks the next one. It is served by the partial index idx_batches_fefo on (medicine_id, expiry_date) WHERE quantity > 0, which only stores batches that still have stock.

Figure 8 traces what happens when two pharmacists attempt to dispense from the same batch at the same instant, and shows how the row lock serialises them.

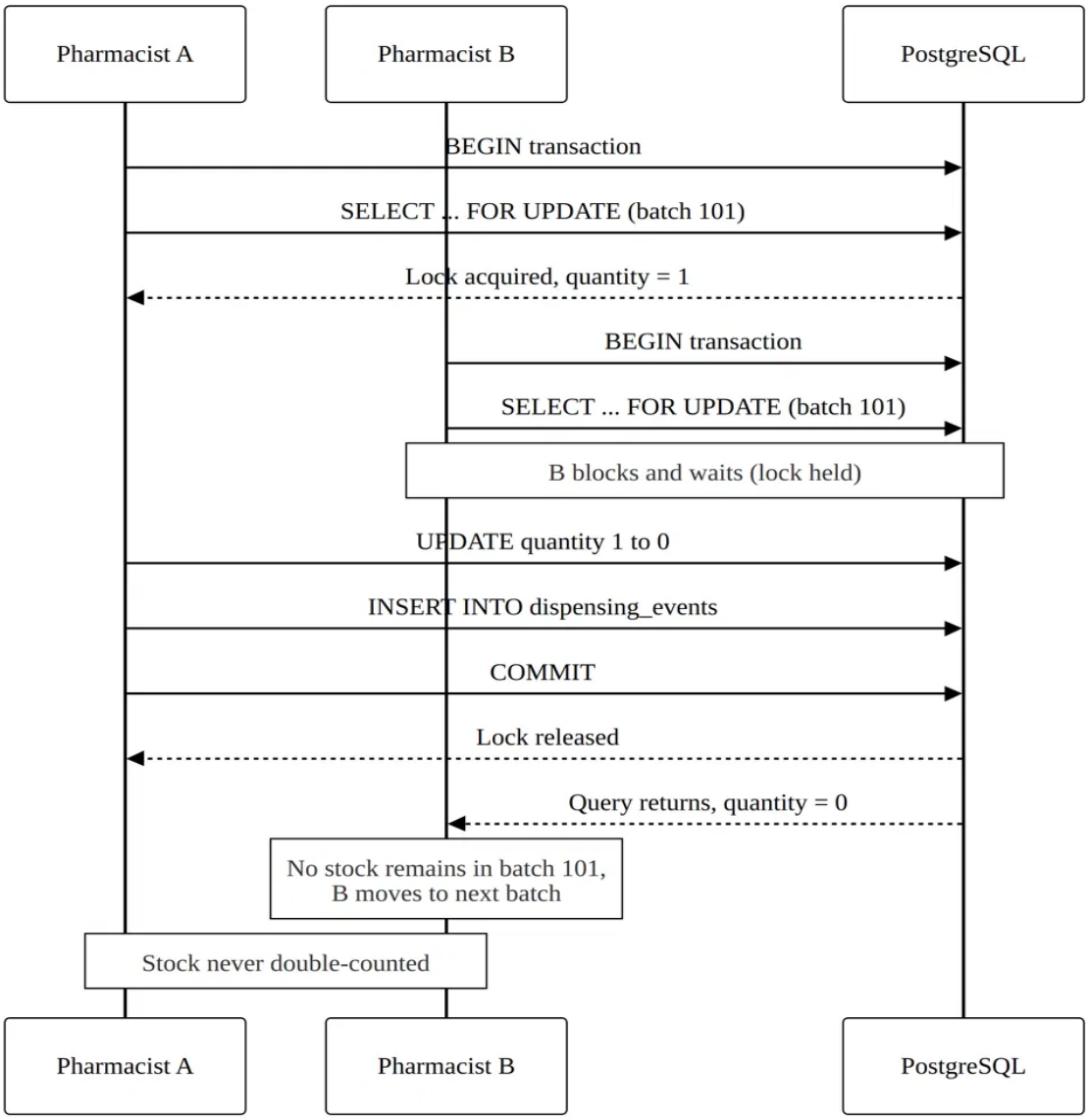


Figure 8: Concurrent dispensing handled by row-level locking (SELECT ... FOR UPDATE)

2.3.2 The Dispensing Event / Audit Log Table

Every dispense, stock adjustment and batch receipt generates an event that has to be recorded. Rather than introducing a separate database engine for this, the log is implemented as an ordinary PostgreSQL table written to as part of, or immediately after, the dispensing transaction commits. The table is deliberately narrow and append-only:

```
CREATE TABLE dispensing_events (
  event_id         BIGSERIAL PRIMARY KEY,
  medicine_id      INT NOT NULL REFERENCES medicines,
  batch_id         INT NOT NULL REFERENCES batches,
  prescription_id  INT REFERENCES prescriptions,
  event_type       VARCHAR(30) NOT NULL CHECK (event_type
                   IN ('DISPENSE','RECEIPT','ADJUSTMENT')),
  quantity_changed INT NOT NULL CHECK (quantity_changed<>0),
performed_by     INT NOT NULL REFERENCES users,
  event_timestamp  TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (event_type <> 'DISPENSE' OR quantity_changed < 0),
  CHECK (event_type <> 'RECEIPT'  OR quantity_changed > 0)
);
```

An index on (medicine_id, event_timestamp DESC) supports the query the system actually asks of this table: “show me the recent history for this medicine, most recent first”, without a full table scan. A secondary index on event_timestamp alone supports time-window reporting such as “most dispensed medicines this week”, supporting reporting queries such as identifying the most dispensed medicines in a given week.

This table is not the source of truth for current stock; the batches table is. The audit log is a record of what happened over time, and because it is written inside the same transaction, it can never disagree with the stock change it describes.

Append-only is enforced by the database itself, twice. The API connects as pharmacy_app, which was granted only SELECT and INSERT on this table, so an UPDATE fails on privilege. The tables are owned by pharmacy_owner, which does have the privilege, so a trigger stops the owner as well:

```
CREATE TRIGGER trg_events_no_update
  BEFORE UPDATE OR DELETE ON dispensing_events
  FOR EACH ROW EXECUTE FUNCTION reject_audit_change();
CREATE TRIGGER trg_events_no_truncate
  BEFORE TRUNCATE ON dispensing_events
  FOR EACH STATEMENT
  EXECUTE FUNCTION reject_audit_change();
```

2.3.3 The Dispense Transaction as Implemented

Figure 9 is the dispense as it runs. Phase A, the preview, reads PostgreSQL and MongoDB and shows the result; nothing is locked or written, so the pharmacist can read the warning or cancel freely. Phase B starts only when Confirm is pressed and is one PostgreSQL transaction lasting a few tens of milliseconds.

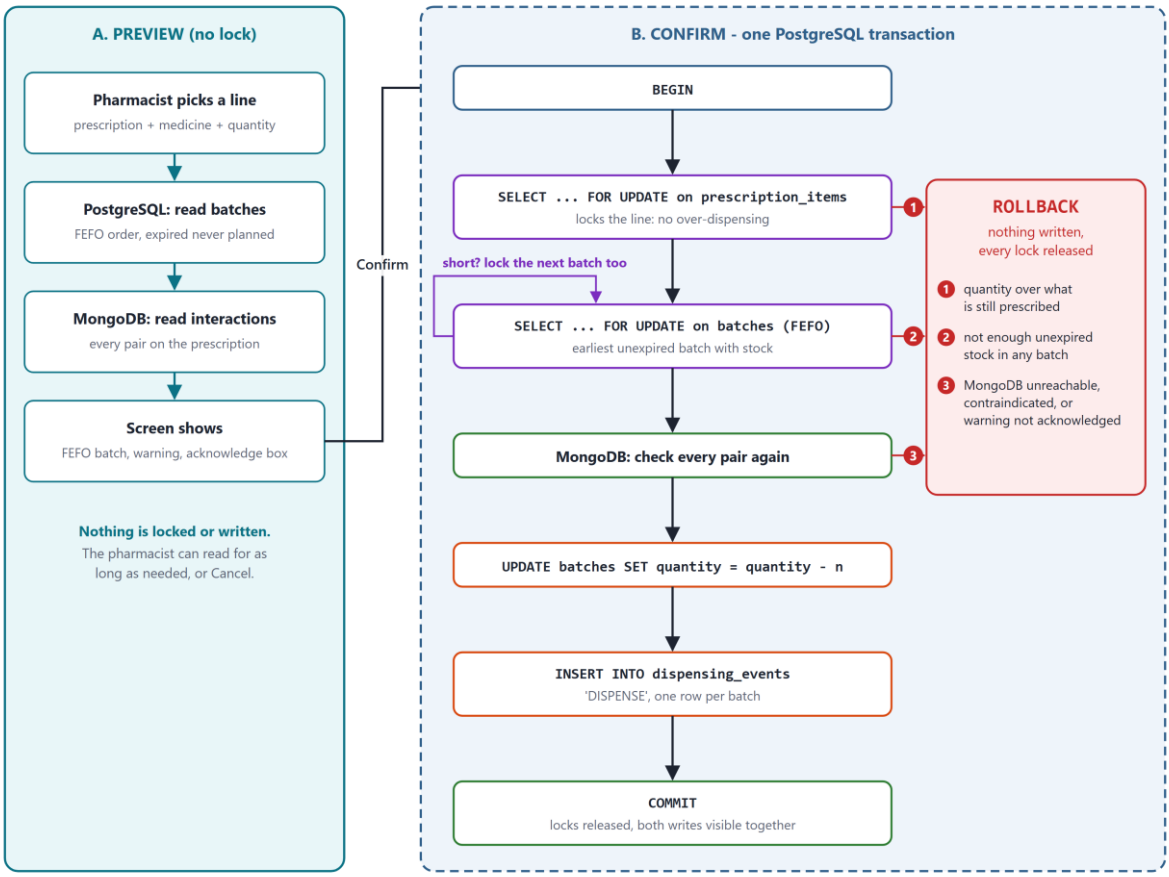


Figure 9: The dispense as implemented: lock-free preview, then one short confirm transaction

Inside the transaction, each step guards one rule:

1. The prescription line is locked first, so two pharmacists cannot both dispense the last ten tablets of one prescription. A quantity above what remains prescribed is refused.
2. The FEFO batch is locked. If it holds less than the request, the next batch by expiry is locked too, and so on; if the unexpired stock is not enough, the transaction is refused.
3. MongoDB is checked again for every pair on the prescription. If MongoDB does not answer within 1.5 seconds the dispense is refused with HTTP 503, because dispensing without the safety check is not allowed. A contraindicated pair is refused outright; a high or moderate one must have been acknowledged on screen.
4. Each batch touched is reduced with UPDATE, and one DISPENSE row per batch is written to dispensing_events.
5. COMMIT makes the stock change and the audit rows visible together. Any refusal above ends in ROLLBACK, which releases every lock and leaves nothing written.

2.4 Database Two - MongoDB: The Safety Knowledge Layer

Why MongoDB here: drug interaction data comes from external medical sources and does not have a fixed shape. Some interactions have dosage thresholds, some do not. Some guidelines have five fields, some have twelve. Forcing this into rigid tables means constant schema migrations. A document model accommodates this variability without requiring schema changes.

- **Schema flexibility.** Each document in the drug_interactions collection can carry different fields depending on what is known about that pair of drugs, without altering a table definition.

- **Document model versus relational.** One interaction record maps to one real-world fact. There is no need to join across multiple tables just to answer "do these two drugs conflict?"

- **Indexing on drug pairs.** A compound index on drug_a and drug_b keeps the interaction check fast, since this query runs on every single dispense.

- **Read-heavy workload.** The collection is updated only occasionally, as new interactions are published, but read on nearly every transaction. MongoDB handles that read pattern well.

2.4.1 Sample Interaction Document

```
{
    drug_a:         "Aspirin",
    drug_b:         "Warfarin",
    severity:       "high",
    effect:         "Increased bleeding risk",
    recommendation: "Avoid combination"
}
```

Figure 10 contrasts this document structure with the equivalent normalised relational representation, illustrating why the document model was preferred for this data.

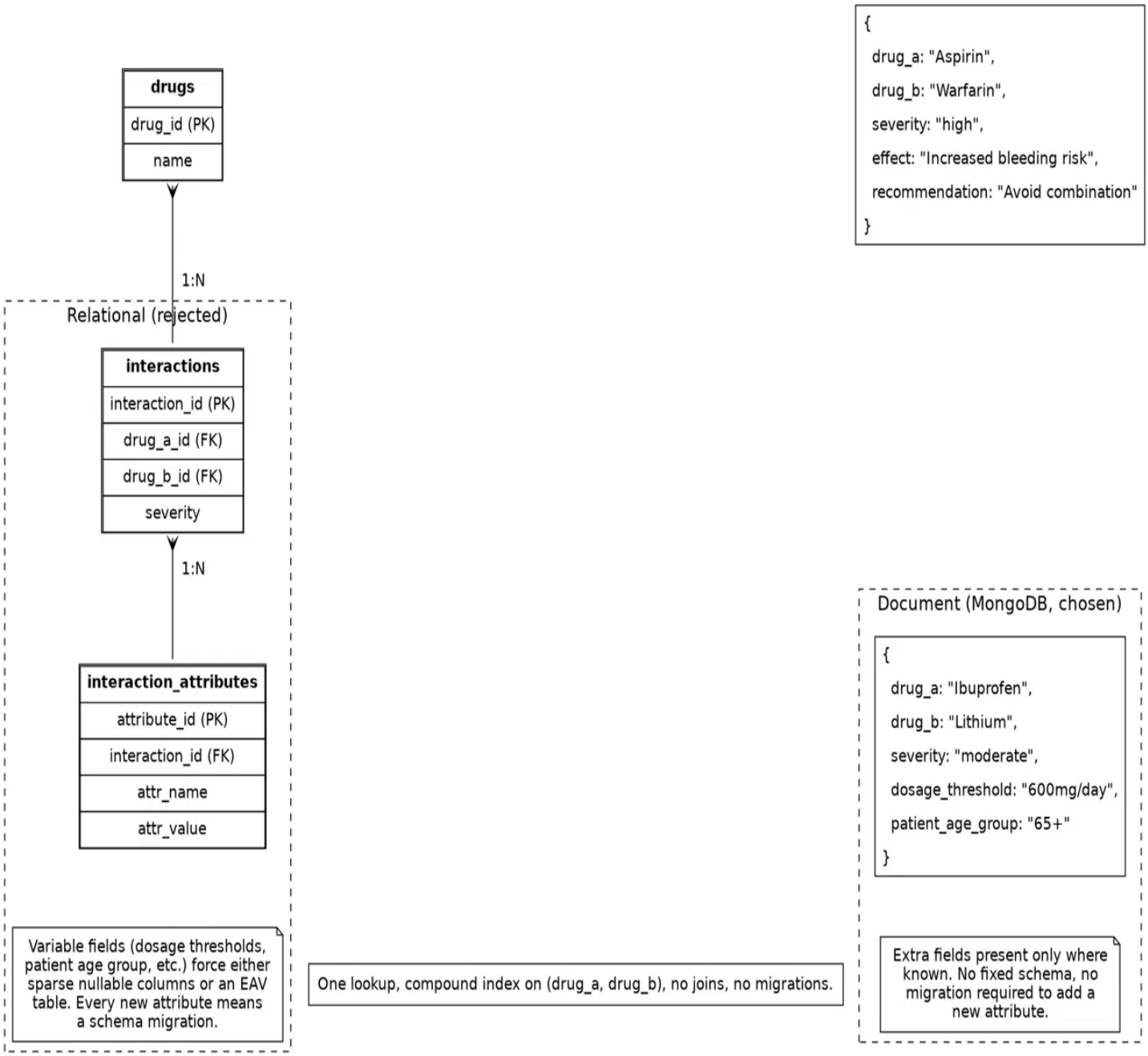


Figure 10: Document model versus relational model for drug interaction data

2.4.2 Interaction Data as Built

The collection holds 30 interaction documents that between them use 18 different field names; no single document has all of them. The same Aspirin and Warfarin pair, as stored, carries two fields the design-review sample did not:

```
{
  drug_a:         "Aspirin",
  drug_b:         "Warfarin",
  severity:       "high",
  effect:         "Increased risk of bleeding",
  recommendation: "Monitor patient closely. Consider
                   dose adjustment or alternative
                   therapy.",
  mechanism:      "Additive antiplatelet and
                   anticoagulant effect",
  monitoring:     ["INR", "signs of bleeding"]
}
```

- **Validator with a floor.** A $jsonSchema validator requires only drug_a, drug_b and a severity of low, moderate, high or contraindicated. Any other field is accepted, so a new kind of guidance needs no migration.
- **One order per pair.** Names are stored in alphabetical order (drug_a < drug_b) and a unique compound index named pair covers (drug_a, drug_b), so one index lookup answers the pair in either order and a pair cannot be stored twice.
- **One query per check.** For a prescription of n medicines the API forms every pair of generic names and asks for all of them in a single $or query; a three-medicine prescription checks three pairs in a few milliseconds.

2.5 Comparison of the Two Databases

The two engines are placed side by side below, on the dimensions that motivated the choice.

Aspect	PostgreSQL	MongoDB
Data shape	Rigid, relational, strongly typed	Flexible, document-oriented (BSON)
Optimised for	Consistency, transactions, locking, referential integrity	Read-heavy access, schema-variable data, fast lookups

Aspect	PostgreSQL	MongoDB
CAP leaning	Consistency first	Consistency with tunable read preference; availability-friendly
Used for	Medicines, batches, prescriptions, stock updates and the dispensing audit log	Drug interactions and clinical guidelines
Write pattern	Transactional writes inside ACID boundaries, plus append-only audit rows	Occasional writes when new interaction data is published
Read pattern	Point lookups and indexed range scans (FEFO, recent history per medicine)	Compound-index lookups on drug pairs, run on nearly every dispense

3. IMPLEMENTATION AND RESULTS

3.1 Changes Since the Design Review

Building the system exposed five places where the design-review version would have been wrong or incomplete. Each was corrected, and each is visible in the demonstration.

Design review	As built	Why
The batch row was locked before the warning was shown (Figure 2).	Preview takes no lock. Locks are taken only inside the short confirm transaction.	A row lock must never wait on a person reading a warning; the transaction now lasts milliseconds.
FEFO picked the earliest expiry with stock.	Expired batches are excluded, shown struck through and counted apart.	The wireframe would have dispensed a batch that had already expired.
One dispense took from one batch.	A larger request continues into the next batch by expiry, one audit row per batch, in the same transaction.	Prescribed quantities often exceed what is left in the oldest batch.
dispensing_events was append-only by convention.	Enforced twice: no UPDATE or DELETE privilege for the API role, and a trigger that stops even the owner.	An audit log the application can edit is not an audit log.
Any interaction showed a warning.	Contraindicated pairs are blocked; high and moderate need acknowledgement; over-dispensing a prescription line is refused.	Not every interaction carries the same risk; some must never be dispensed.

3.2 Database Objects as Built

- **Tables.** Six, as in the design: medicines, batches, prescriptions, prescription_items, users and dispensing_events, with NOT NULL, CHECK, UNIQUE and foreign-key constraints on every one.
- **Indexes.** idx_batches_fefo (partial, for FEFO), idx_events_medicine_time and idx_events_time (history and reports), idx_items_prescription, and lower-case indexes on medicine name and generic name for search.
- **Views.** v_stock, v_near_expiry, v_top_week and v_prescription_progress, so reports are computed on read.

- **Trigger and roles.** reject_audit_change() behind two triggers; pharmacy_owner owns the schema and pharmacy_app, the role the API uses, holds only the privileges it needs.
- **MongoDB.** The drug_interactions collection with its $jsonSchema validator and the unique pair index.
- **Seed data.** 44 medicines, 181 live batches plus 60,000 emptied historical batches (so indexes have real work to do), 10 prescriptions, 3,181 audit events and 30 interaction documents. The audit log reconciles exactly with stock.

3.3 Measured Results

Measured on the seeded database on a laptop, from the screens in Section 4.

Test	Result	What it shows
Two pharmacists, last unit	B waited from 258 ms to 2,019 ms, took the lock one millisecond after A committed, then took the next batch	The row lock serialises the two; stock is never double-counted
MongoDB unreachable	Refused with HTTP 503 and ROLLBACK after 1,533 ms	No dispense happens without the safety check
FEFO query, EXPLAIN ANALYZE	0.037 ms with the partial index, 7.205 ms with index scans off (195 times faster); 3 buffers read against 443	Only 180 of 60,182 batches have stock, and the index stores only those
Audit log rewrite	pharmacy_app: refused, 42501 permission denied. pharmacy_owner: refused by the trigger	Append-only is enforced by the database
End-to-end checks	smoke.js: 24 of 24 pass	Every rule above holds on every run

3.4 Running the Project

The whole system starts from the project folder. setup.js creates both databases and loads the seed, and can be re-run at any time to restore the seeded state; npm run build compiles the React app; npm start serves the API and the app on http://localhost:4000; npm run smoke runs the 24 end-to-end checks.

```
npm install
node scripts/setup.js
npm run build
npm start
npm run smoke
```

4. DEMONSTRATION

This section walks through every feature in the order it is demonstrated, with a screenshot of each step. All screenshots were taken from the running system on the seeded data.

4.1 Creating a Prescription

The Dispense screen opens with both databases reported up in the header. "+ New prescription" opens the form.

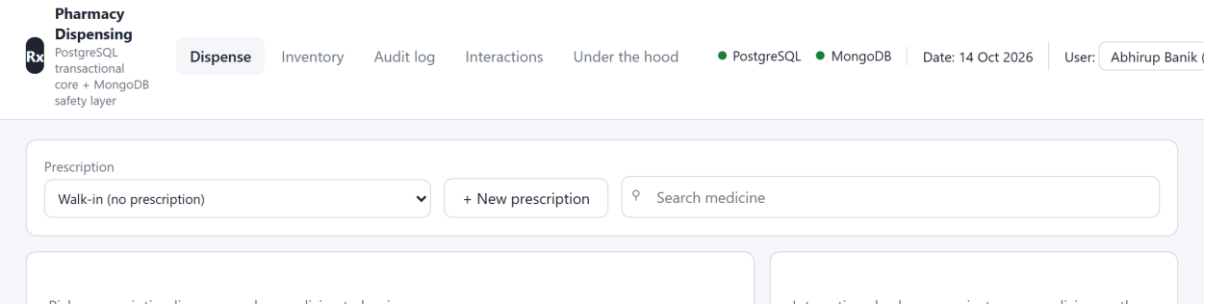


Figure 11: The Dispense screen at start; the header shows PostgreSQL and MongoDB both reachable

A new patient, Kavya Rao, aged 52, is entered with two medicines: Warfarin 5 mg (10 tablets) and Fluconazole 150 mg (1 tablet).

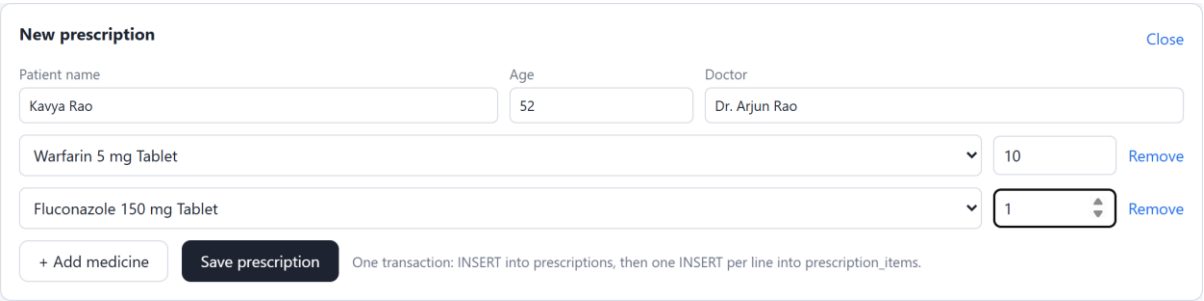


Figure 12: New prescription form filled in

Saving inserts one prescriptions row and one prescription_items row per medicine in a single transaction. A blank name or a missing medicine is rejected by CHECK and foreign-key constraints and nothing is half-written.

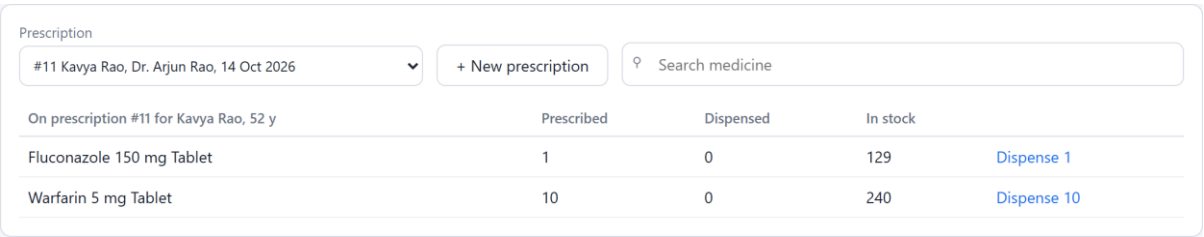


Figure 13: Prescription #11 saved and opened, with stock for each line

Picking the Fluconazole line runs the preview. MongoDB finds the Fluconazole and Warfarin interaction on this new prescription immediately.

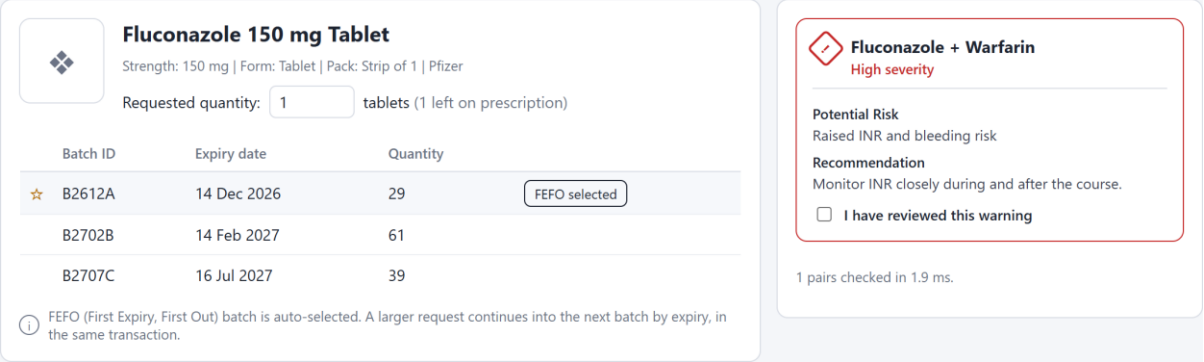


Figure 14: Preview of the new prescription: FEFO batch selected and a high-severity warning

4.2 Dispensing with an Interaction Warning

Prescription #1 (Ramesh Iyer) holds Aspirin, Paracetamol and Warfarin. Picking Aspirin shows the batches in expiry order: the expired batch is struck through and never offered, and

the earliest unexpired batch, B2611A, is starred. The Aspirin and Warfarin warning is high severity, so Confirm stays disabled until it is acknowledged.

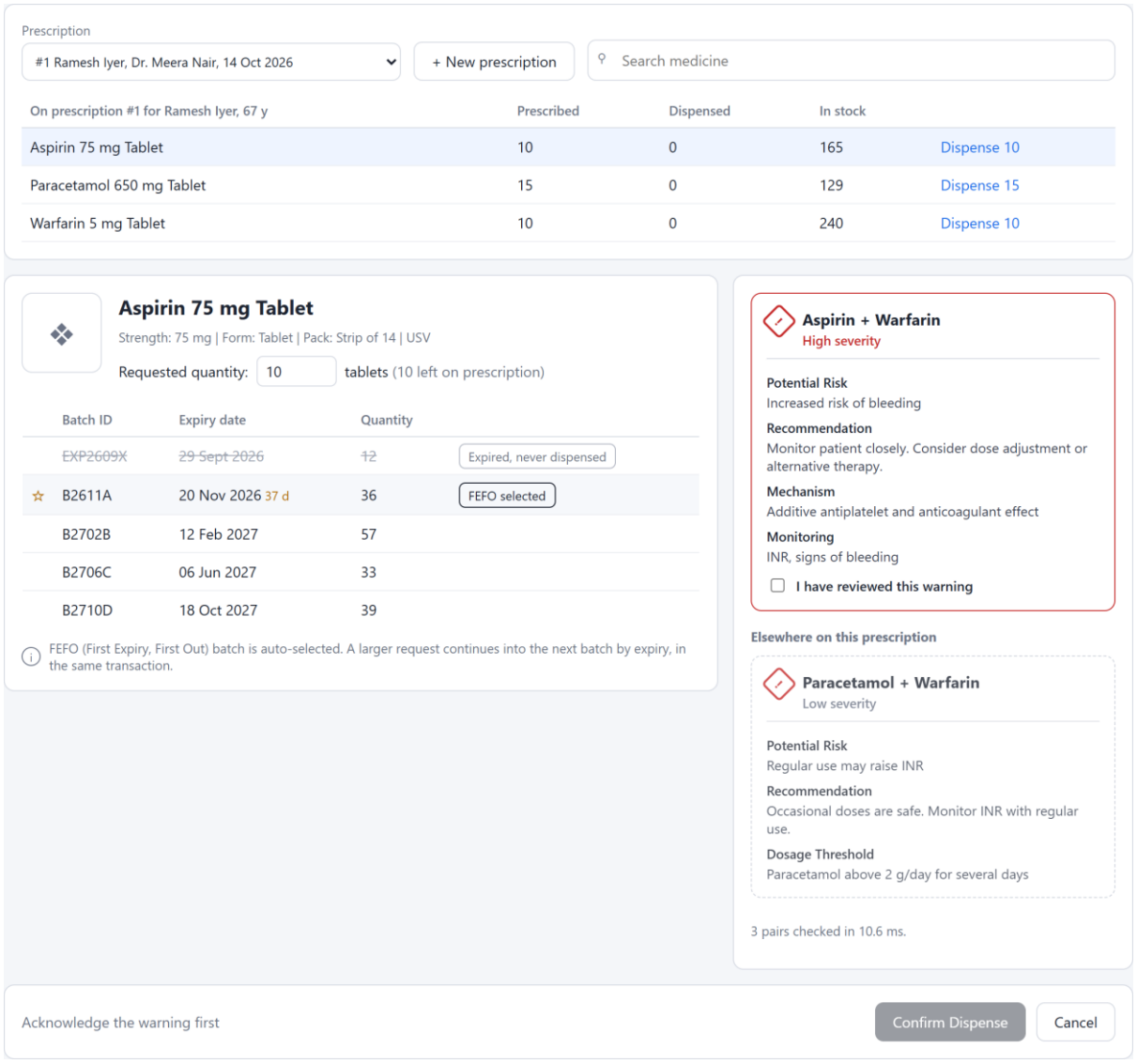


Figure 15: Preview: FEFO batch, expired batch excluded, warnings for every pair on the prescription

After the box is ticked, Confirm runs the transaction. The trace lists each statement with its time: BEGIN, both locks, the MongoDB check, the UPDATE, the audit INSERT and COMMIT.

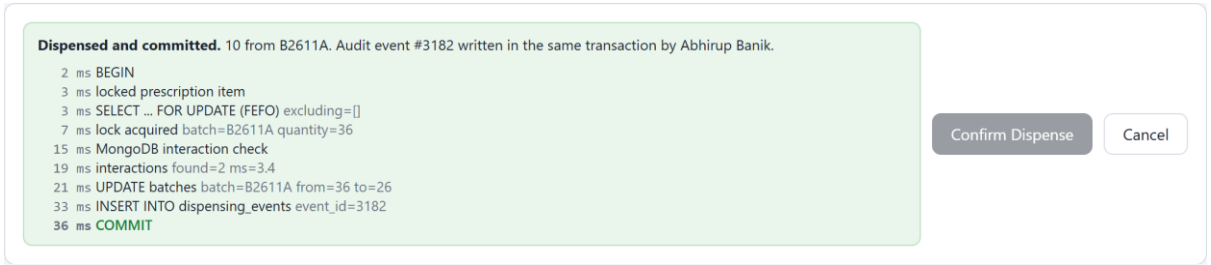


Figure 16: Committed: 10 tablets from B2611A, audit event written in the same transaction

The prescription now shows Aspirin as dispensed and the stock reduced from 165 to 155.

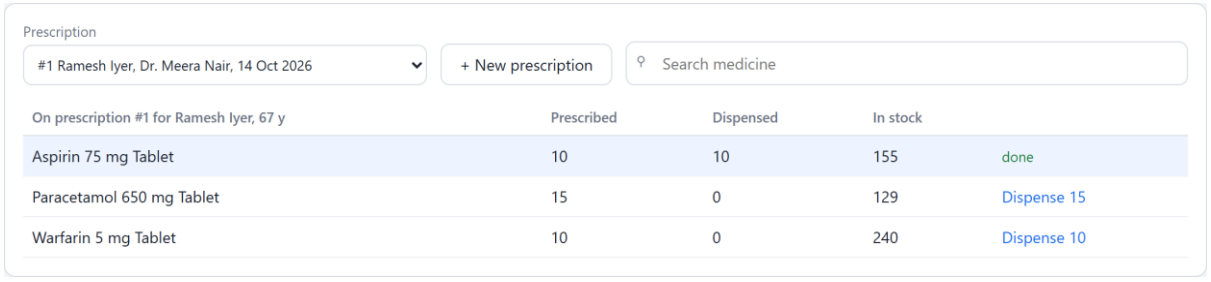


Figure 17: The prescription after the dispense

4.3 Unsafe Dispenses Are Refused

The Warfarin line on the same prescription raises the same warning from the other side, so it also needs acknowledgement. Without it the server refuses with HTTP 409 and rolls back even if the button is bypassed.

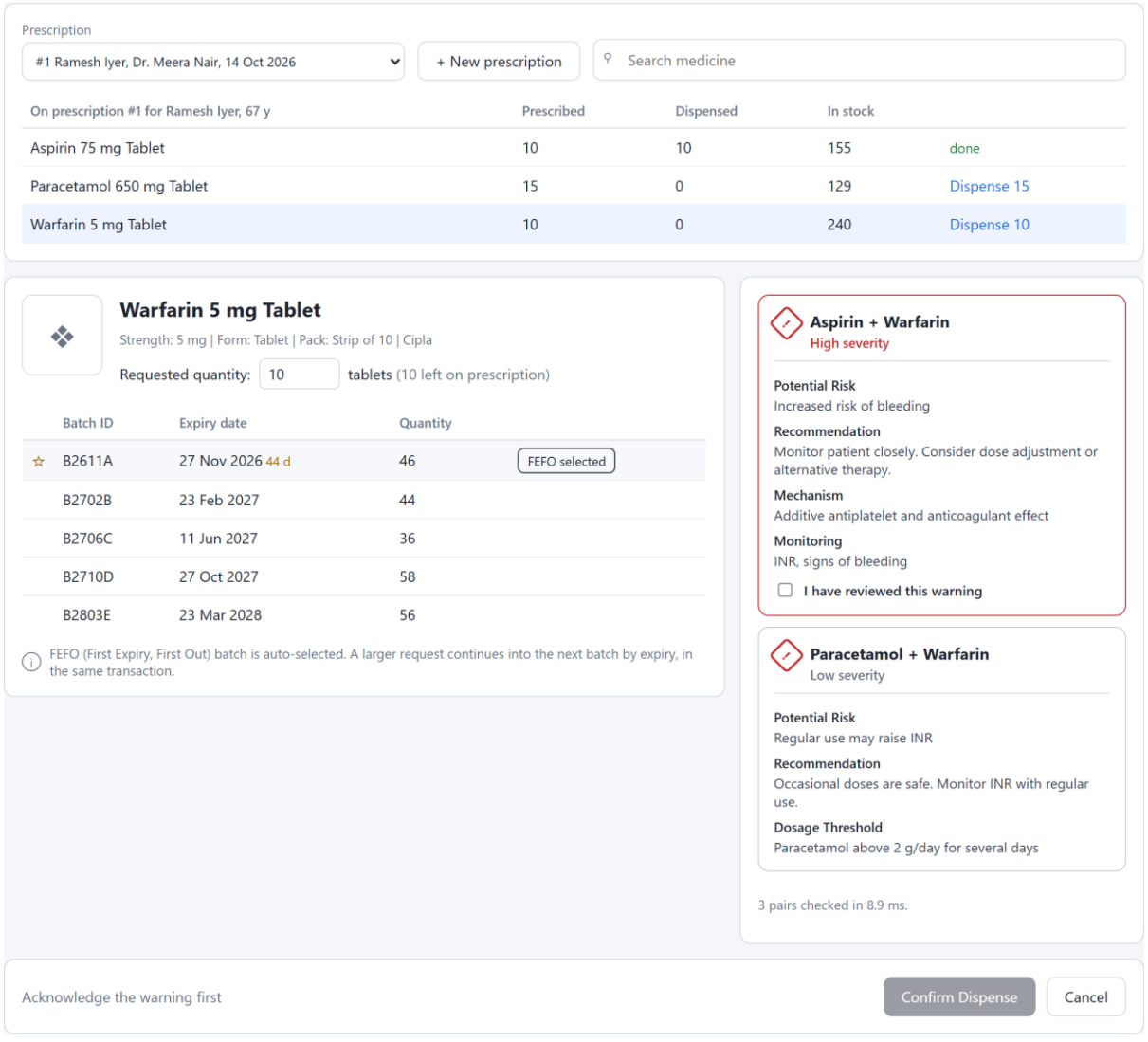


Figure 18: A high-severity warning that has not been acknowledged blocks Confirm

Prescription #4 combines Sildenafil with Isosorbide Mononitrate. The pair is contraindicated, so there is no box to tick: the dispense is blocked outright.

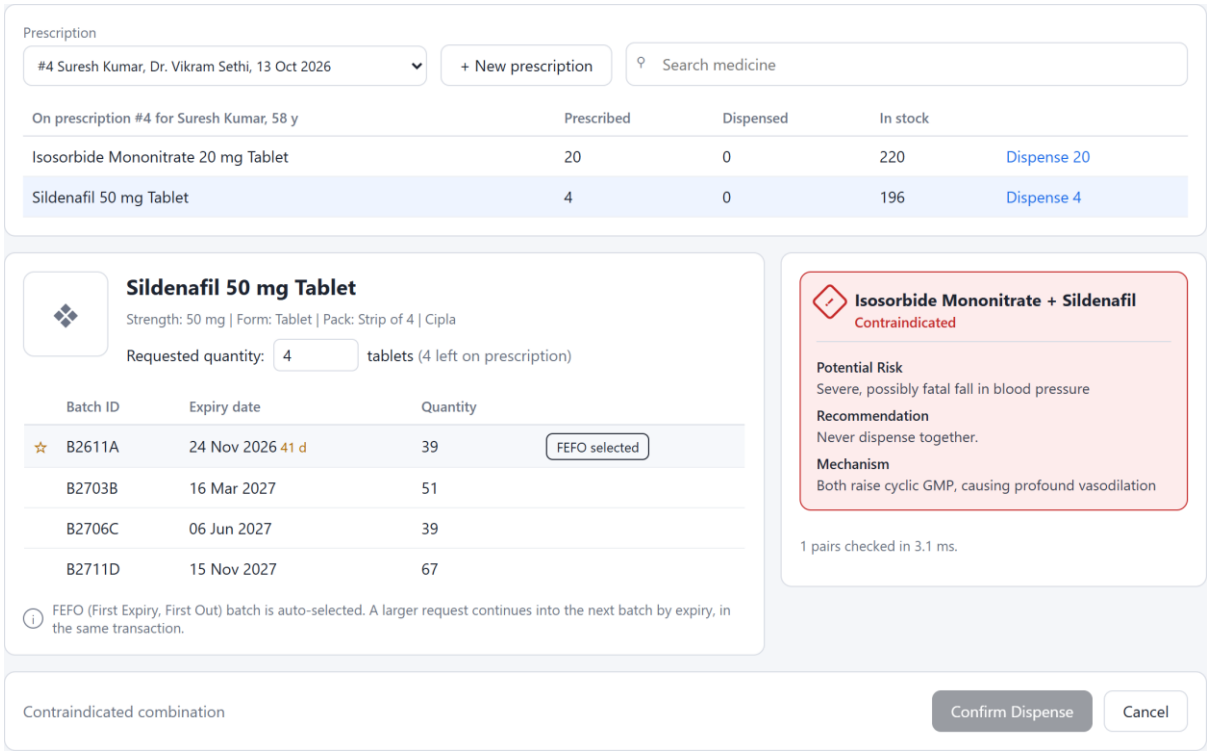


Figure 19: A contraindicated combination cannot be dispensed

4.4 A Request Larger Than One Batch

32 tablets of Montelukast are requested while the FEFO batch holds 22. The preview shows the plan: 22 from B2612A, then 10 from B2703B.

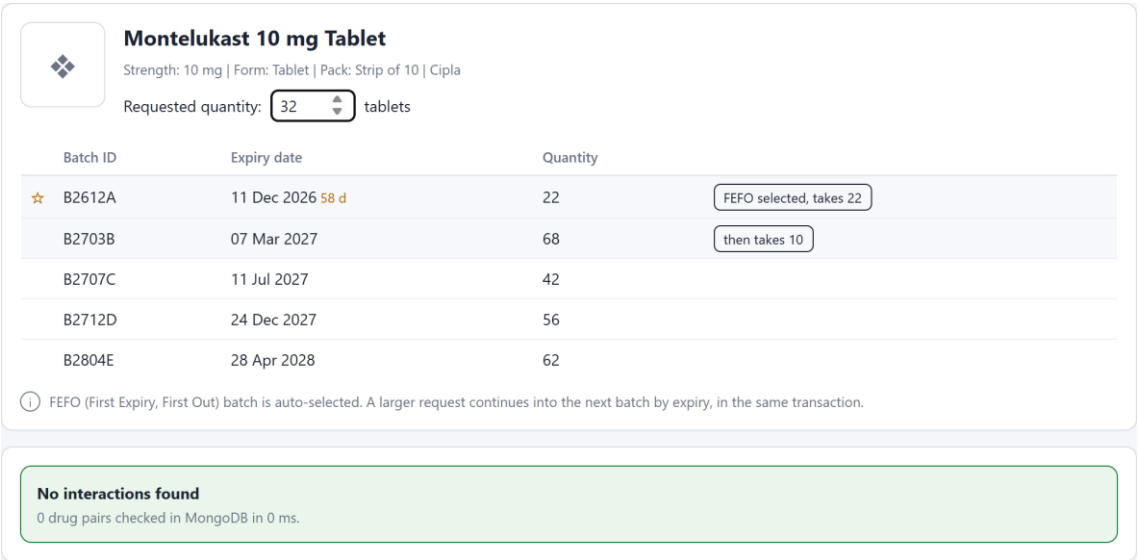


Figure 20: The plan spans two batches in expiry order

Confirm locks both batches in that order, updates both and writes two audit rows, all in one transaction.

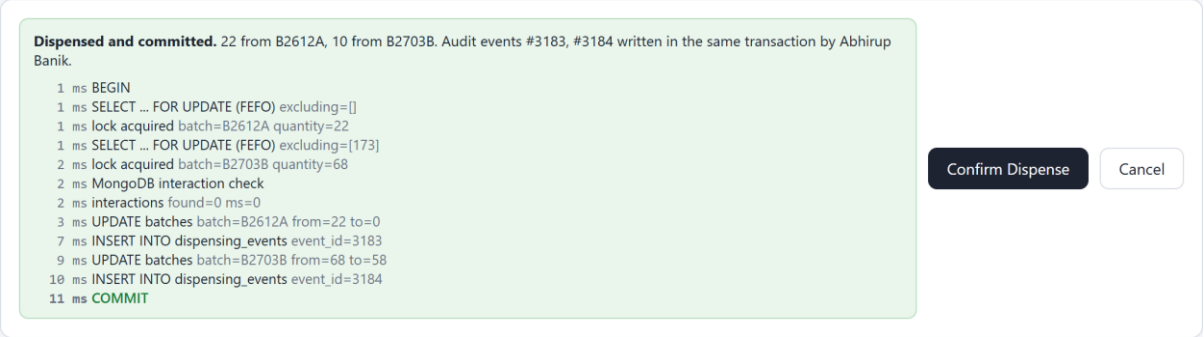


Figure 21: Committed across two batches with one audit event per batch

4.5 Receiving Stock and the Inventory View

On the Inventory screen a new Paracetamol batch, RCV2611P, expiring 31 March 2028, is received with 120 units. A CHECK constraint rejects an expiry date before the date received.

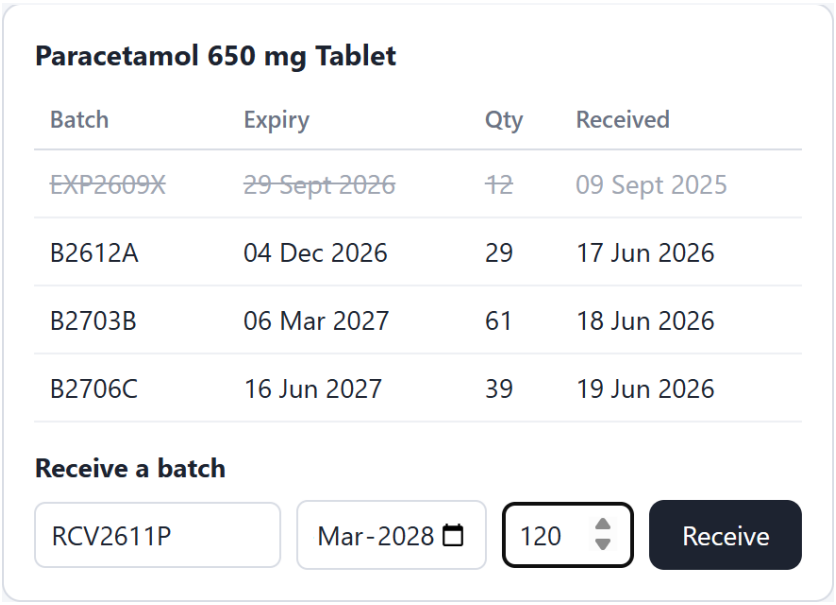


Figure 22: Receiving a new batch

The batch row and a RECEIPT audit row are written in one transaction, and the batch appears in the list.

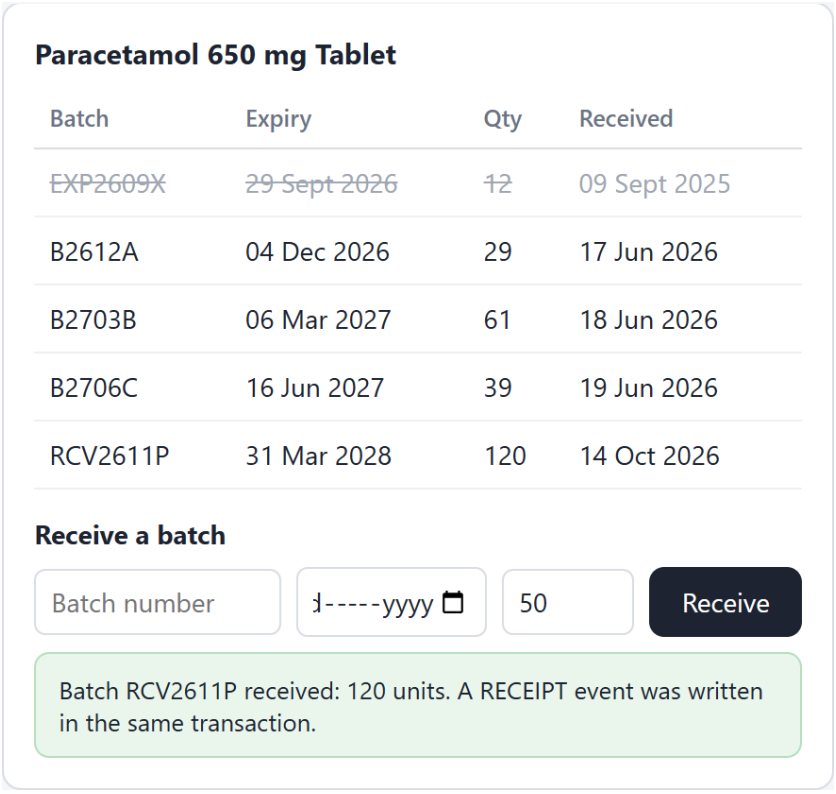


Figure 23: Batch received and listed

Stock by medicine comes from the v_stock view. Expired units are counted in their own column and never offered for dispensing.



Stock by medicine

Filter

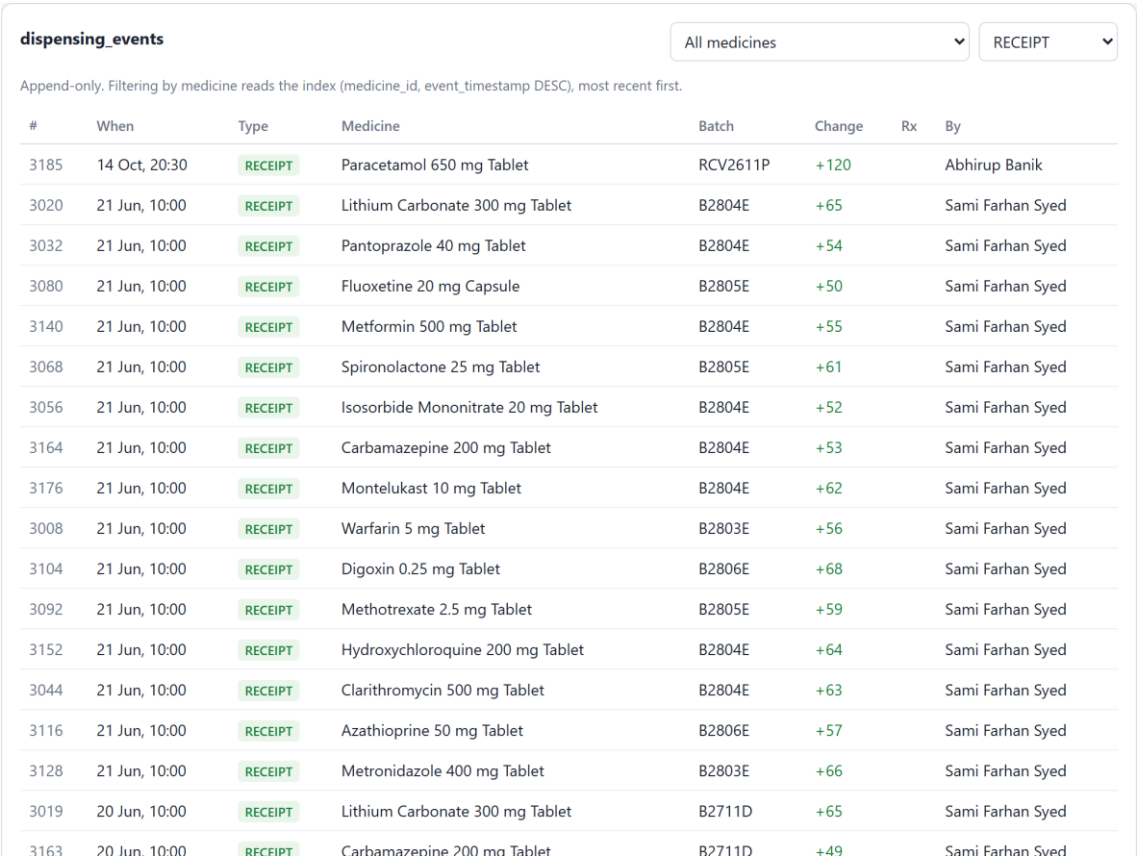
From the view v_stock. Expired units are counted apart and never offered for dispensing.

Medicine	Usable	Expired	Next expiry	Batches
Allopurinol 100 mg Tablet	196	0	19 Dec 2026	4
Amiodarone 200 mg Tablet	180	0	28 Nov 2026	4
Amlodipine 5 mg Tablet	126	12	17 Nov 2026	4
Amoxicillin 500 mg Capsule	166	0	04 Dec 2026	4
Aspirin 75 mg Tablet	155	12	20 Nov 2026	5
Atorvastatin 10 mg Tablet	180	0	13 Dec 2026	4
Azathioprine 50 mg Tablet	225	0	16 Nov 2026	5
Azithromycin 500 mg Tablet	147	0	16 Dec 2026	3
Calcium Carbonate 500 mg Tablet	182	0	30 Nov 2026	4
Carbamazepine 200 mg Tablet	225	0	20 Nov 2026	5
Cetirizine 10 mg Tablet	126	0	27 Nov 2026	3
Ciprofloxacin 500 mg Tablet	135	0	08 Dec 2026	3
Clarithromycin 500 mg Tablet	215	0	20 Dec 2026	5
Clopidogrel 75 mg Tablet	138	0	15 Nov 2026	3
Digoxin 0.25 mg Tablet	270	0	05 Dec 2026	5
Fluconazole 150 mg Tablet	129	0	14 Dec 2026	3
Fluoxetine 20 mg Capsule	220	0	03 Dec 2026	5
Furosemide 40 mg Tablet	141	0	12 Dec 2026	3
Glimepiride 2 mg Tablet	198	0	21 Dec 2026	4
Hydroxychloroquine 200 mg Tablet	250	0	09 Dec 2026	5
Ibuprofen 400 mg Tablet	178	12	11 Dec 2026	5
Isosorbide Mononitrate 20 mg Tablet	220	0	01 Dec 2026	5

Figure 24: Inventory from v_stock: usable and expired units, next expiry, number of batches

4.6 The Audit Log and the Weekly Report

Filtering dispensing_events to RECEIPT shows the batch just received at the top, with who received it and when.



dispensing_events

All medicines | RECEIPT

Append-only. Filtering by medicine reads the index (medicine_id, event_timestamp DESC), most recent first.

#	When	Type	Medicine	Batch	Change	Rx	By
3185	14 Oct, 20:30	RECEIPT	Paracetamol 650 mg Tablet	RCV2611P	+120		Abhirup Banik
3020	21 Jun, 10:00	RECEIPT	Lithium Carbonate 300 mg Tablet	B2804E	+65		Sami Farhan Syed
3032	21 Jun, 10:00	RECEIPT	Pantoprazole 40 mg Tablet	B2804E	+54		Sami Farhan Syed
3080	21 Jun, 10:00	RECEIPT	Fluoxetine 20 mg Capsule	B2805E	+50		Sami Farhan Syed
3140	21 Jun, 10:00	RECEIPT	Metformin 500 mg Tablet	B2804E	+55		Sami Farhan Syed
3068	21 Jun, 10:00	RECEIPT	Spironolactone 25 mg Tablet	B2805E	+61		Sami Farhan Syed
3056	21 Jun, 10:00	RECEIPT	Isosorbide Mononitrate 20 mg Tablet	B2804E	+52		Sami Farhan Syed
3164	21 Jun, 10:00	RECEIPT	Carbamazepine 200 mg Tablet	B2804E	+53		Sami Farhan Syed
3176	21 Jun, 10:00	RECEIPT	Montelukast 10 mg Tablet	B2804E	+62		Sami Farhan Syed
3008	21 Jun, 10:00	RECEIPT	Warfarin 5 mg Tablet	B2803E	+56		Sami Farhan Syed
3104	21 Jun, 10:00	RECEIPT	Digoxin 0.25 mg Tablet	B2806E	+68		Sami Farhan Syed
3092	21 Jun, 10:00	RECEIPT	Methotrexate 2.5 mg Tablet	B2805E	+59		Sami Farhan Syed
3152	21 Jun, 10:00	RECEIPT	Hydroxychloroquine 200 mg Tablet	B2804E	+64		Sami Farhan Syed
3044	21 Jun, 10:00	RECEIPT	Clarithromycin 500 mg Tablet	B2804E	+63		Sami Farhan Syed
3116	21 Jun, 10:00	RECEIPT	Azathioprine 50 mg Tablet	B2806E	+57		Sami Farhan Syed
3128	21 Jun, 10:00	RECEIPT	Metronidazole 400 mg Tablet	B2803E	+66		Sami Farhan Syed
3019	20 Jun, 10:00	RECEIPT	Lithium Carbonate 300 mg Tablet	B2711D	+65		Sami Farhan Syed
3163	20 Jun, 10:00	RECEIPT	Carbamazepine 200 mg Tablet	B2711D	+49		Sami Farhan Syed

Figure 25: Audit log filtered to receipts, beside the most dispensed medicines this week

Filtering to Aspirin reads the index on (medicine_id, event_timestamp DESC): the dispense from Section 4.2 is the newest row, followed by the history.

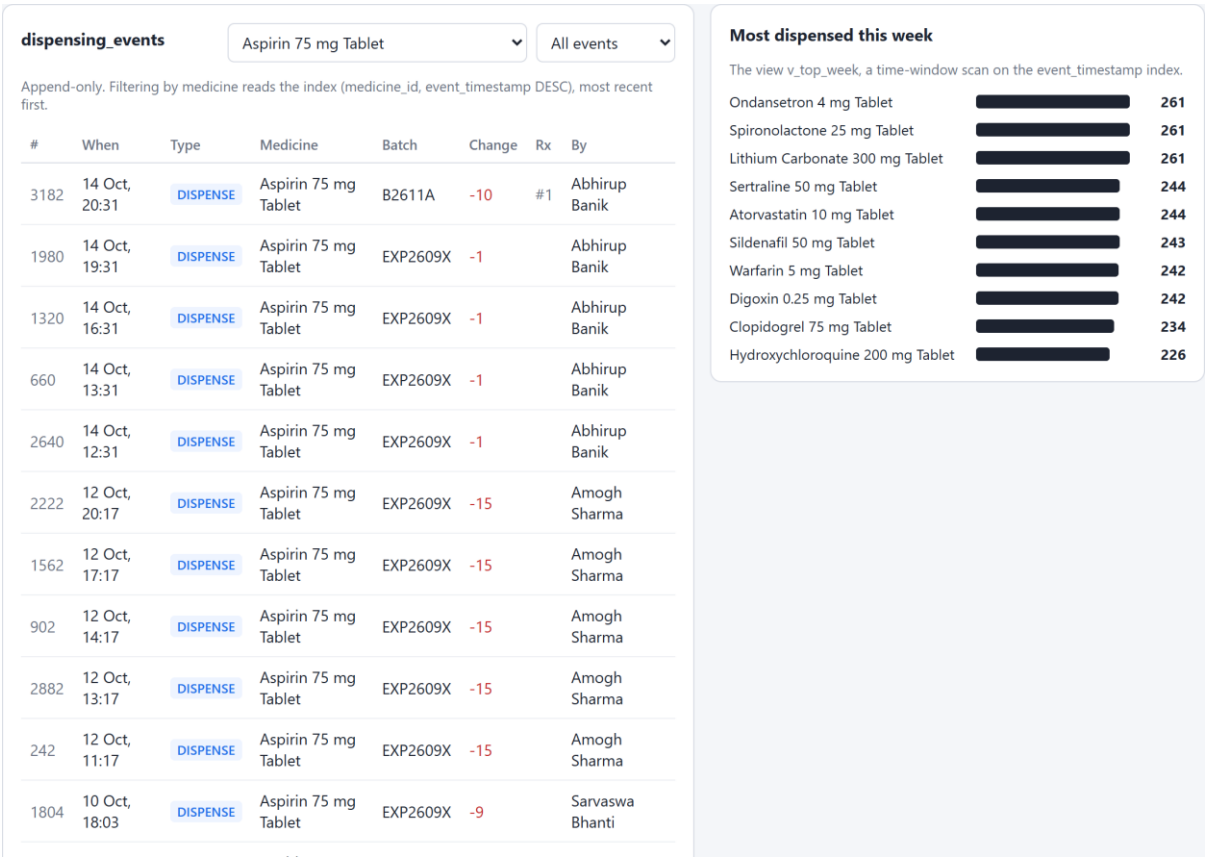


Figure 26: History of one medicine, most recent first

4.7 Drug Interactions in MongoDB

The Interactions screen lists every document. The documents carry different fields: dose_adjustment on one, clinical_note and intended_combination on another, separate_by_hours on a third.

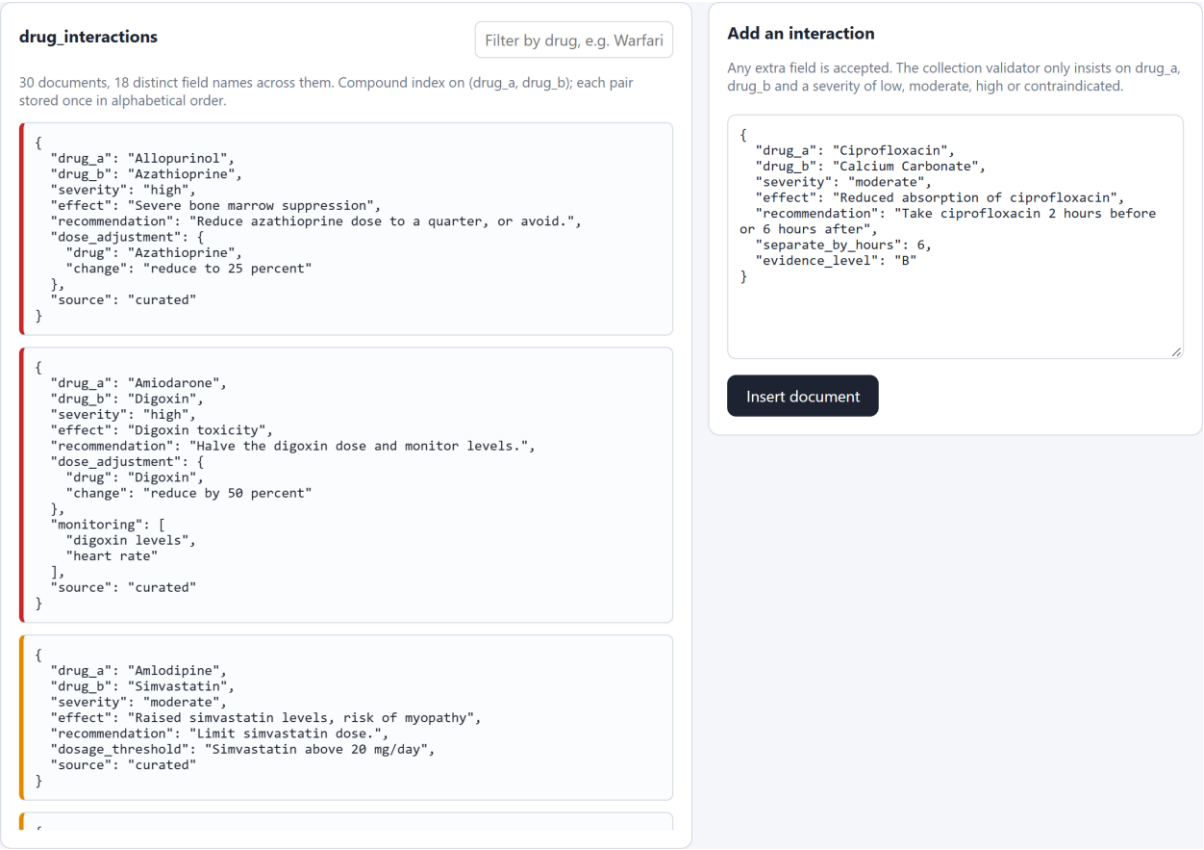


Figure 27: The drug_interactions collection: one document per pair, fields vary by pair

A new interaction is inserted with two fields no other document has, separate_by_hours and evidence_level. No ALTER TABLE and no migration is needed.

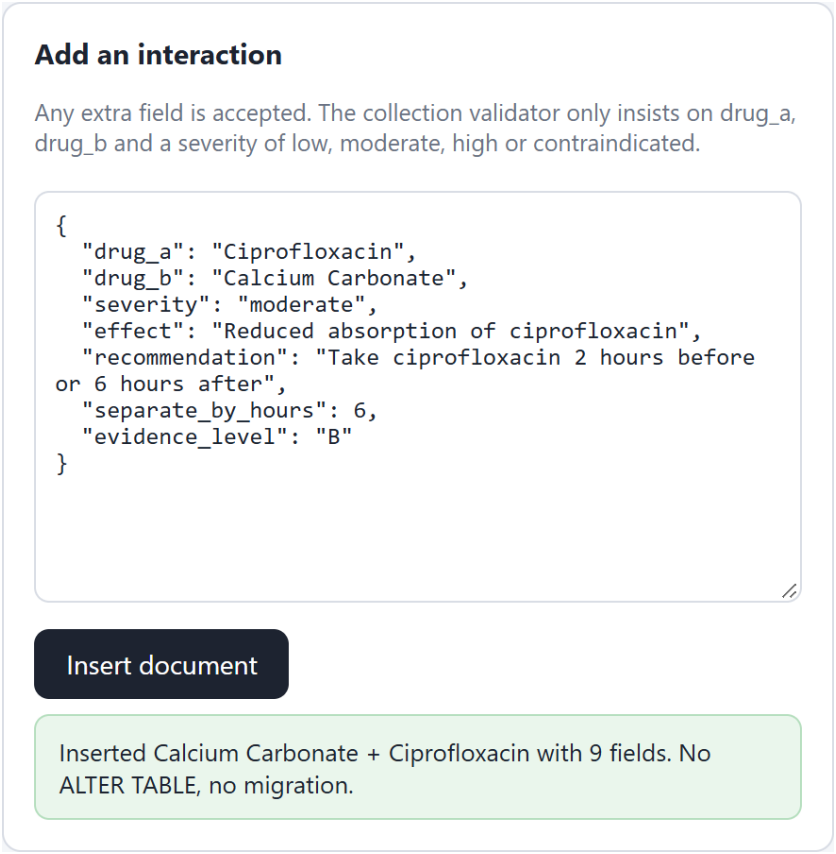


Figure 28: A document with new fields is accepted

A document without a severity is rejected by the collection validator, so flexibility still has a floor.

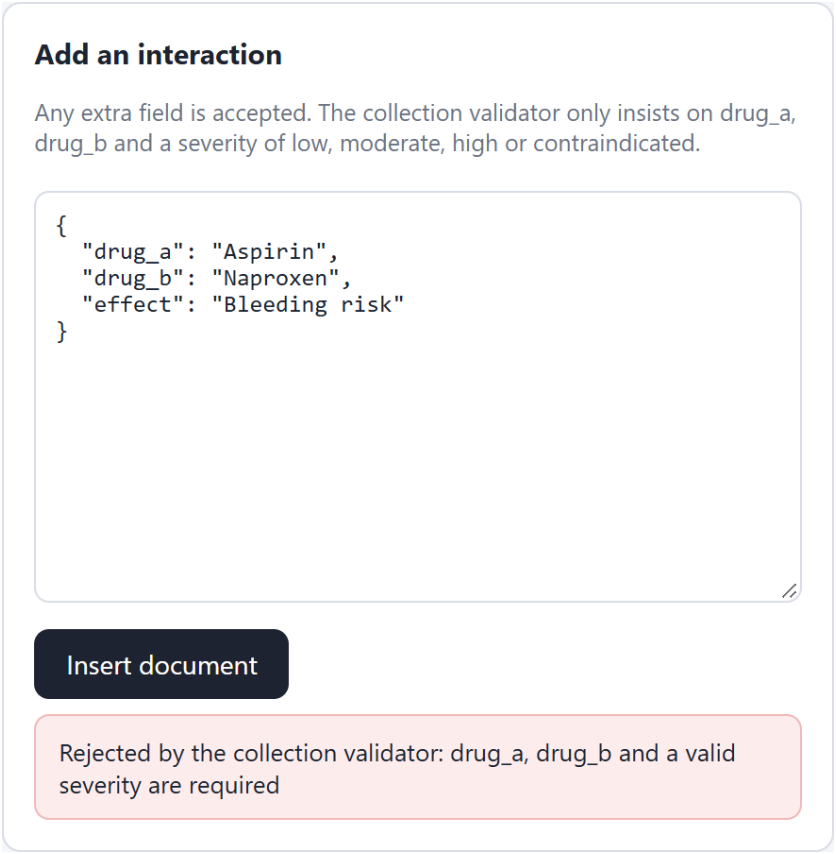


Figure 29: The $jsonSchema validator rejects a document missing a required field

4.8 Two Pharmacists, the Last Unit

The oldest Cetirizine batch is set to exactly one unit. Pharmacist A locks it and holds the lock for two seconds; pharmacist B asks for the same unit 250 ms later. B waits on the row lock,

takes it only after A commits, sees the batch is now empty and takes the next batch. Both succeed and the unit is sold once. This is Figure 8 running live.

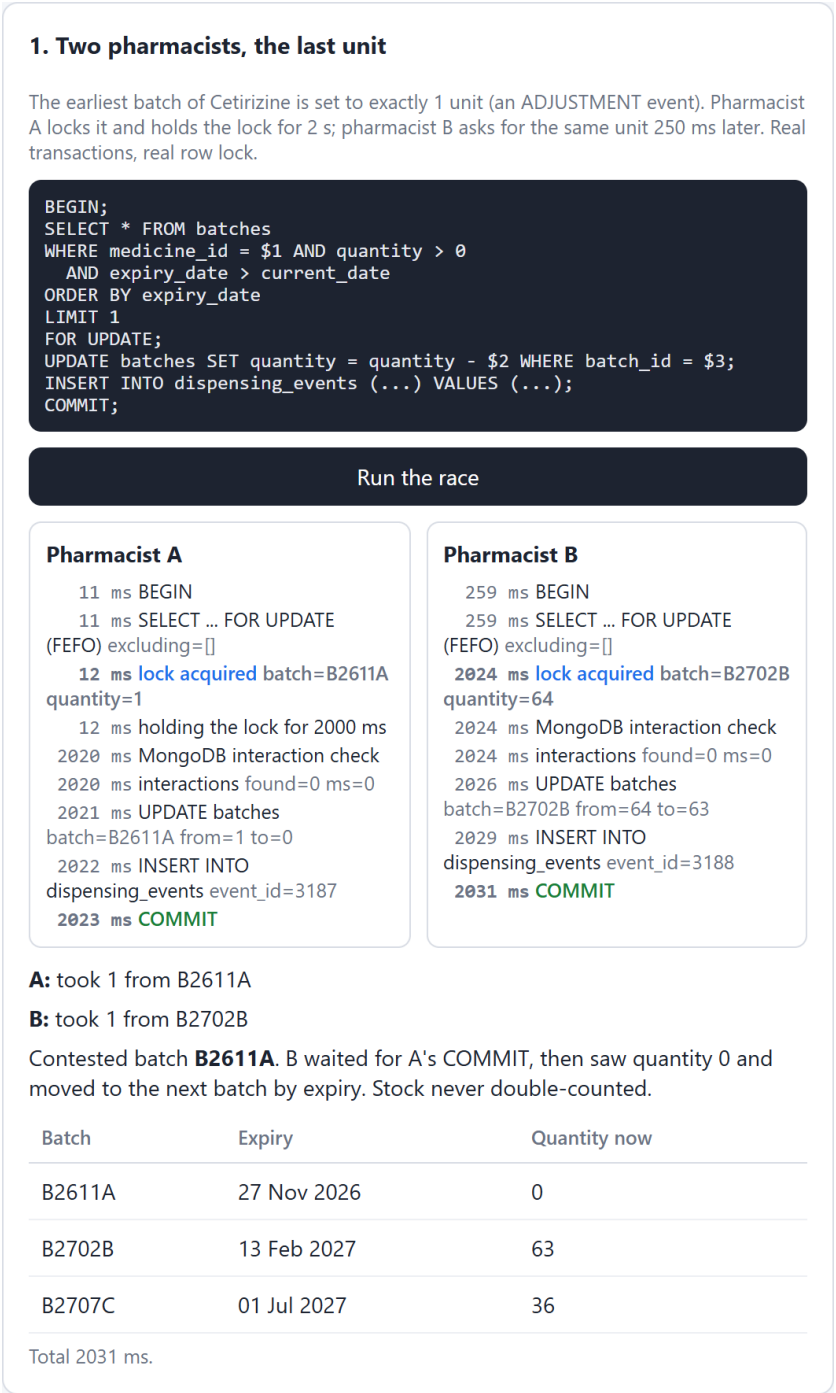


Figure 30: Two concurrent dispenses serialised by SELECT ... FOR UPDATE

4.9 MongoDB Unreachable Mid-Dispense

"Cut MongoDB" points the driver at a closed port, exactly as if the server were down. The dispense reaches the MongoDB check inside the transaction, times out after 1.5 seconds and rolls back.

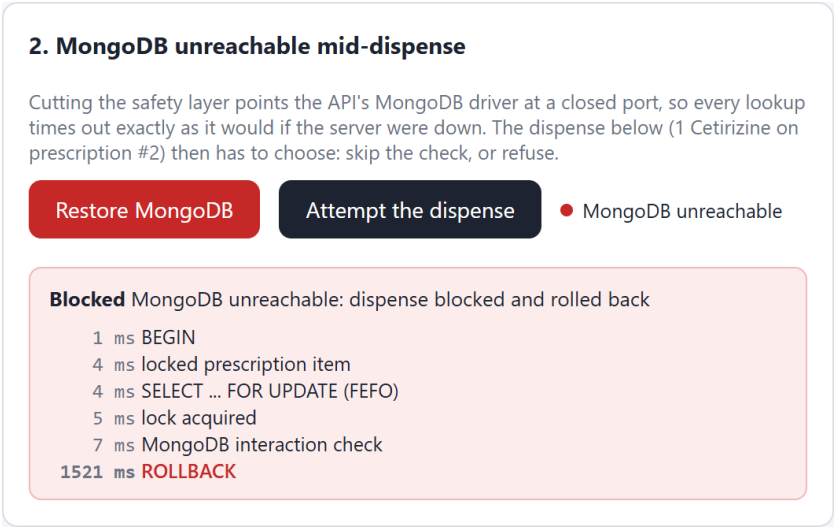


Figure 31: With MongoDB unreachable the dispense is blocked and rolled back

After "Restore MongoDB" the same dispense commits normally.

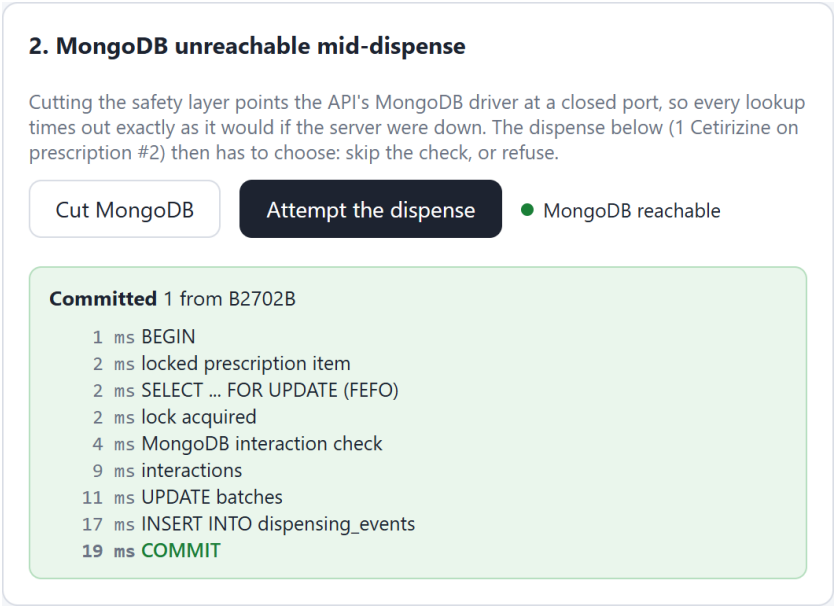


Figure 32: With MongoDB back, the same dispense commits

4.10 Why the FEFO Query Stays Fast

EXPLAIN ANALYZE runs the FEFO query twice: with the partial index, and with index scans switched off for one transaction. The index reads 3 buffers; the sequential scan reads 443 and filters out 60,178 rows.

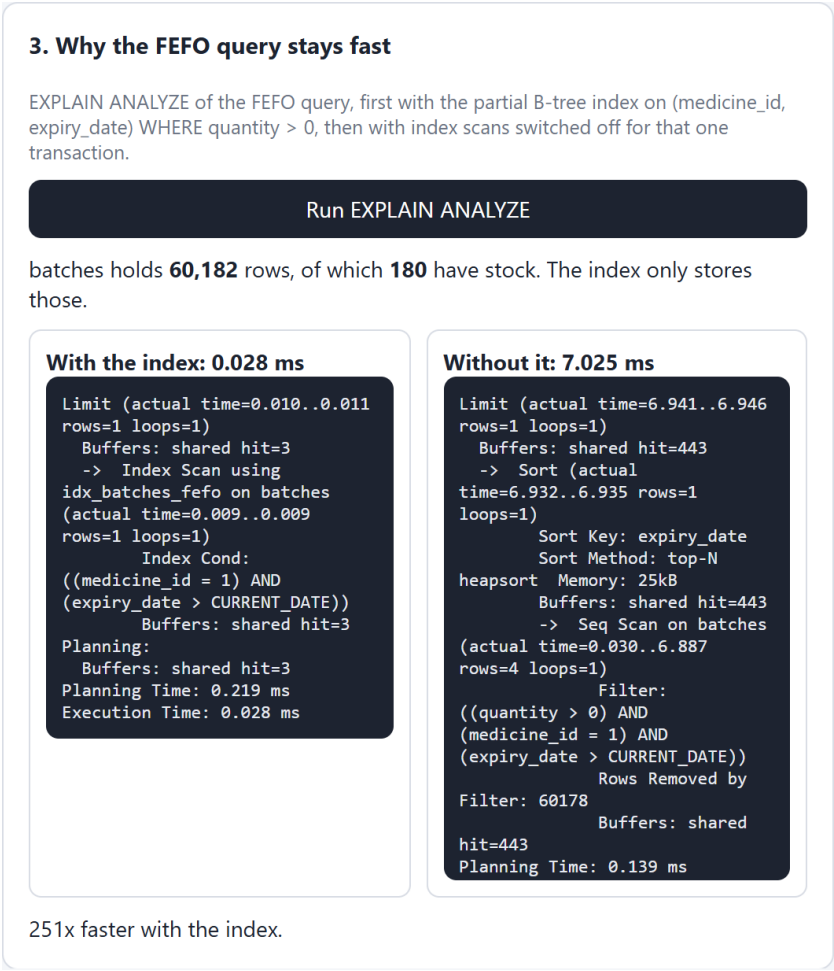


Figure 33: EXPLAIN ANALYZE: index scan on idx_batches_fefo against a sequential scan

4.11 The Audit Log Cannot Be Rewritten

The same UPDATE on the newest audit row is tried as the API role and as the table owner. The first fails on privilege (42501); the second has the privilege but is stopped by the trigger.

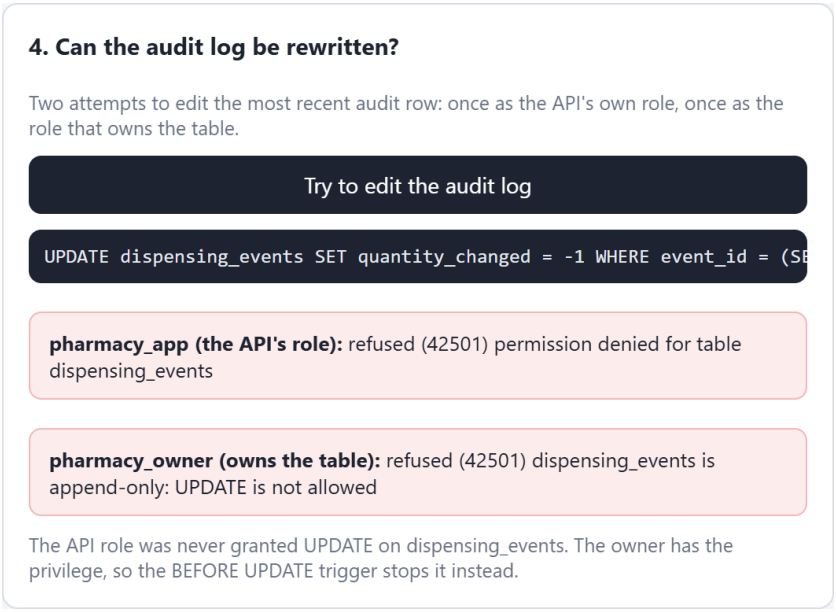


Figure 34: Both attempts to edit dispensing_events are refused